UNIDAD 3.

CONCEPTOS AVANZADOS



Tabla de contenido

Cadenas....5

Combinar código HTML con código PHP....5

Variables dentro de cadenas....5

Uso de la sentencia echo....5

Uso de la sintaxis <?= ... ?>....6

Fragmentos de HTML dentro de estructuras condicionales....7

Distribución del código....8

Manipulación de cadenas....9

Acceder a los caracteres de una cadena....9

strlen() y mb_strlen()....10

*mb_strpos() y strp*os()....10

*mb_substr() s*ubstr()....11

trim()....11

str_replace()....12

number_format()....13

Arrays....16

Arrays indexados numéricamente....16

Inicializar un array....16

Acceder a un elemento de un array....17

Recorrer un array....17

Ordenar un array....19

Arrays no secuenciales....19

Arrays asociativos....19

Arrays no homogéneos....21

Arrays multidimensionales....21

Mostrar el contenido de un array....21

Desbordamiento de arrays....22

Sentencia foreach....22

PHP 8. Nueva Función: array_is_list....22

Comprobación y eliminación de posiciones de Arrays....23

isset()....23

unset()....24

Funciones....25

Funciones definidas por el usuario....25

Declaración de una función....25

Llamada a una función....26

Devolución de un valor....26

Argumentos....26
Paso de argumentos por valor....27
Paso de argumentos por referencia....27
Alcance de las variables....28
Creación de bibliotecas....29
Las instrucciones include() y require()....30
require_once e include_once....31
Otros tipos de funciones....31
Extensiones de PHP....31
Funciones. Novedades incorporadas en php....33
Tipado fuerte en PHP: Descriptivo y Estricto....33
Tipado fuerte descriptivo....33
Tipado fuerte Estricto....34
Listas de argumentos de longitud variable ¶....34
Union Types....35
Expresiones match, ¿mejora del switch?....35
Nuevo tipo mixed....36
Sintaxis First-class Callable function....37
Nuevo tipo de retorno never....37
Formularios....38
Formularios HTML....38
Declaración del formulario....38
Campos de los formularios....38
Botones de los formularios....39
Recepción de datos de formularios....40
Páginas que envían, reciben y procesan datos....41
Redirección de páginas web....43
Nuevos Operadores Ternarios....45
1. El Operador Ternario Tradicional (? :)....45
2. Operador de Fusión de Null (??)....46
Fusion null. Más de dos operandos....47
3, 4, 5 o más operandos....47
Ejemplo final....48
Fusión de Null Compuesta (??=)....48
3. Operador Seguro contra Nulos (?->)....48
Un poco de Seguridad en el Envío de Datos. Sanitización de datos.Limpiar y validar variables GET y POST en PHP....49
¿Cuál es la solución para estos problemas?....49
Validación y desinfección de datos con filtros....49

Lista de filtros de sanitización..... 52
Lista de filtros de Validación..... 52
Vamos a lo práctico: filter_var_array..... 54
Subida de ficheros al Servidor.....57
Array Superglobal $_FILES..... 58
Subida de múltiples ficheros..... 58
Más ejemplos..... 59

Cadenas.

El manejo de cadenas en *PHP* es de vital importancia, ya que la única forma que tenemos de comunicarnos con el cliente es a través de texto.

Además, hay que tener en cuenta que el producto de un programa en *PHP* es una página web con formato *HTML*. Es decir, un fichero de texto.

Combinar código HTML con código PHP.

Como en una página web dinámica programada en *PHP* hay código de dos tipos, lo mejor es mantenerlos bien diferenciados. Hay que ser especialmente cuidadoso, procurando que el código sea muy legible. Esto nos facilitará el posterior mantenimiento de la página, y hará mucho más sencillo dotarla de elementos de diseño.

El objetivo ideal sería que un diseñador web pudiera aplicar diseño a una página *PHP*, sin tener que preocuparse de la programación que contiene. En niveles más avanzados se utilizan sistemas de plantillas que ayudan notablemente en la tarea de separar el código *HTML* y *PHP*.

Variables dentro de cadenas.

Hasta ahora, siempre que hemos necesitado mostrar un texto compuesto por una cadena y el valor de una variable los hemos tenido que unir usando el operador concatenar, ".". Pero *PHP* ofrece una sintaxis alternativa, que resulta más compacta y puede ser de utilidad en muchos casos.

Ejemplo 3.1.1. Si ponemos una variable en el interior de una cadena entre comillas, *PHP* la interpreta, y en lugar del nombre de la variable escribe su valor. Lo podemos comprobar a continuación, donde los dos fragmentos de código producen los mismos resultados.

```php
<?php
  $edad = 46;
  echo "Belinda tiene " . $edad . " años.";
?>
```

```php
<?php
  $edad = 46;
  echo "Belinda tiene $edad años.";
?>
```

Uso de la sentencia echo.

Es recomendable usar esta forma de producir la salida cuando es predominante la cantidad de código *PHP* sobre la cantidad de código *HTML*. Lo mismo se aplica al uso de la función *print()* que produce los mismos resultados.

Hay partidarios de usar una forma y partidarios de usar la otra. Independientemente de cual se use, no es muy recomendable mezclarlas dentro del mismo programa.

Ejemplo 3.1.2. En este pequeño programa predomina el código *PHP* sobre el código *HTML*.

```html
<html>
  <head>
    <title>Números primos</title>
  </head>
```

```
  <body>
<?php
  define("LIMITE", 100);
  echo "Los números primos del 1 al " . LIMITE . " son:";
  echo "<br>";
  for ($i = 1; $i <= LIMITE; $i++) {
    $es_primo = TRUE; //Hipótesis inicial, es primo
    for ($j =2; $j < $i; $j++) {
      if(($i % $j) == 0) { //Tiene un divisor
        $es_primo = FALSE; //Luego no es primo
        break;
      }
    }
    if ($es_primo) {
        echo "$i"; echo "<br>";
    }
  }
?>
  </body>
</head>
```

Uso de la sintaxis <?= ... ?>.

A la inversa, cuando predomina el código *HTML* sobre el código *PHP* puede ser interesante mostrar la salida usando esta sintaxis. De cualquier modo, debemos tener presente que esta etiqueta no está disponible en todos los servidores web, y podemos encontrarnos algunos casos en que, debido a la configuración del propio servidor, esta etiqueta no funcione adecuadamente.

Ejemplo 3.1.3. En este pequeño programa predomina el código *HTML*sobre el código *PHP*.

```
<html>
 <head>
   <title>El hombre del tiempo</title>
   </
  head>
<body bgcolor="#cccccc">
  <table border="1">
   <tr>
     <td align="center" colspan="2">
       Hola, soy el hombre del tiempo.<br>
       Mi método para adivinar el clima es infalible.
     </td>
   </tr>
   <tr>
     <td align="center">
       Temperatura mínima prevista
     </td>
     <td align="center">
       Temperatura máxima prevista
     </td>
   </tr>
   <tr>
     <td align="center">
       <font size="+2"><?= rand(0, 15)?></font> grados.
```

```
      </td>
      <td align="center">
        <font size="+2"><?= rand(15, 30)?></font> grados.
      </td>
    </tr>
     </
  table>
</html>
```

Fragmentos de HTML dentro de estructuras condicionales.

Cuando tenemos que mostrar trozos de código *HTML* muy grandes, y éste se encuentra dentro de una estructura condicional, puede ser muy pesado construirlos a base de sentencias *echo*.

En este caso es preferible volver a cerrar el bloque de código *PHP* con la etiqueta "?>", escribir el código *HTML* que se ejecuta condicionalmente, y abrir otra vez el código *PHP* con la etiqueta "<?php" para terminar la estructura condicional.

De este modo, reducimos la mezcla e interferencia entre etiquetas y sentencias de ambos lenguajes, lo que repercute en una mayor claridad y sencillez. Además, este sistema combinado con el uso de la sintaxis "<=? ... ?>" produce un código compacto y fácil de leer.

Ejemplo 3.1.4. Aquí se ven las dos estrategias. La segunda es más clara y comprensible.

```
<?php
  $autorizado = FALSE;
  if (!$autorizado) {
    echo '<center>';
    echo '<table bgcolor="#cccccc">';
    echo '<tr>';
    echo '<td align="center">';
    echo '<font color="#ff0000" size="+2">';
    echo 'No está autorizado a ver esta página.';
    echo '<br>';
    echo 'Contacte con el administrador del sistema.';
    echo    '</font>';
    echo '</td>';
       echo   '</tr>';
    echo   '</table>';
    echo '</center>';
  }
  echo '<br>';
?>

<?php
  $autorizado = FALSE;
   if (!$autorizado) {
?>
<center>
  <table bgcolor="#cccccc">
   <tr>
     <td align="center">
       <font color="#ff0000" size="+2">
```

```
        No está autorizado a ver esta
        página.<br>  Contacte  con  el
        administrador del sistema.
      </font>
    </td>
  </tr>
 </table>
</center>
<?php
 }
?>
```

Distribución del código.

La última cuestión que nos queda por resolver es donde debemos situar el código. Salvo el código que muestra resultados por pantalla, que tiene que aparecer en el lugar apropiado de la página, el código *PHP* se puede situar en cualquier lugar del fichero. La mejor opción es separar en la medida de lo posible el código encargado de:

- Inicializar las constantes y variables.
- Leer datos de entrada.
- Realizar operaciones.

Este código es conveniente ponerlo al principio de la página. Los resultados de las operaciones que se quieran mostrar se almacenan entonces en variables auxiliares y se muestran en el cuerpo de la página HTML donde corresponda haciendo uso de las etiquetas “<?= … ?>”.

Ejemplo 3.1.5. Esta es una página web en *PHP* bien construida. Usa un algoritmo para hallar el máximo común divisor de dos números generados aleatoriamente.

```
<?php
  // Inicialización
  DEFINE("LIMITE", 100);
  // Lectura de datos
  $i = rand(1, LIMITE); // Simulamos la lectura
  $j = rand(1, LIMITE);
  // Operaciones con los datos
  if ($i > $j) {
     $mayor = $i;
     $menor = $j;
  } else {
     $mayor = $j;
     $menor = $i;
  }
  while (($mayor % $menor) != 0) {
    $auxiliar = $menor;
    $menor = $mayor % $menor;
    $mayor = $auxiliar;
  }
  $mcd = $menor;
?>
<html>
  <head>
    <title>Máximo comun divisor</title>
```

```
  </head>
  <body>
    El máximo común divisor de <?= $i ?> y <?= $j ?> es <?= $mcd ?>.
  </body>
</html>
```

Esto no es posible siempre. Por ejemplo, si tenemos que mostrar una lista de 11 resultados el bucle que los recorre deberá estar mezclado con el código *HMTL*. De todas formas con la experiencia iremos aprendiendo poco a poco a extraer la mayor parte del código *PHP* al comienzo de la página.

Manipulación de cadenas.

PHP dispone de funciones predefinidas en el núcleo que nos permiten hacer operaciones con los textos almacenados en cadenas. Ya hemos visto las función *echo* y *print()* pero hay unas cuantas más.

Cadenas en php

Podemos considerar que en php es un string cualquier cadena de caracteres entrecomilladas.

```
$cadenaDoble =" Hola que tal ";
$cadenaSimple = 'Muy bien y tu ';
```

Si bien en principio no existe ninguna diferencia entre comilla entre usar cadenas simples y dobles en realidad hay pequeños matices de cierta importancia.

1. Cadena simples: Se consideran más eficientes.
2. Cadenas dobles: Si un string está delimitado con comillas dobles ("), PHP interpretará las siguientes secuencias de escape como caracteres especiales:

Secuencia	Significado
\n	avance de línea (LF o 0x0A (10) en ASCII)
\r	retorno de carro (CR o 0x0D (13) en ASCII)
\t	tabulador horizontal (HT o 0x09 (9) en ASCII)
\v	tabulador vertical (VT o 0x0B (11) en ASCII) (desde PHP 5.2.5)
\e	escape (ESC o 0x1B (27) en ASCII) (desde PHP 5.4.4)
\f	avance de página (FF o 0x0C (12) en ASCII) (desde PHP 5.2.5)
\\	barra invertida
\$	signo de dólar
\"	comillas dobles
\[0-7]{1,3}	la secuencia de caracteres que coincida con la expresión regular es un carácter en notación octal, que silenciosamente desborda para encajar en un byte (p.ej. "\400" === "\000")
\x[0-9A-Fa-f]{1,2}	la secuencia de caracteres que coincida con la expresión regular es un carácter en notación hexadecimal
\u{[0-9A-Fa-f]+}	la secuencia de caracteres que coincida con la expresión regular es un punto de código de Unicode, la cual será imprimida al string como dicha representación UTF-8 del punto de código (añadido en PHP 7.0.0)

Acceder a los caracteres de una cadena.

Se puede acceder a los `caracteres` de las cadenas indicando la posición del carácter que queremos extraer. Las posiciones van desde 0 hasta el número de caracteres (longitud) menos uno. Si

intentamos acceder a una posición más allá del final de la cadena nos devolverá la cadena vacía (cadena sin ningún carácter = "").

Ejemplo 3.1.6. Se muestran varias pruebas de extracción de caracteres de una cadena.

```php
<?php
  $cadena = "Pernambuco";
  echo "Cadena a analizar: '$cadena'<br>";

  // Caracter en la primera posición
  echo "Caracter en la posición 0: '$cadena[0]'<br>";

  // Caracteres de posiciones intermedias
  echo "Caracter en la posición 1: '$cadena[1]'<br>";
  echo "Caracter en la posición 5: '$cadena[5]'<br>";
  echo "Caracter en la posición 7: '$cadena[7]'<br>";

  // La cadena tiene longitud 10, la última posición es la 9
  echo "Caracter en la posición 9: '$cadena[9]'<br>";

  // La cadena tiene longitud 10, en la posición 12 no hay nada
  echo "Caracter en la posición 12: '$cadena[12]'<br>";
?>
```

strlen() y mb_strlen().

Por lo general no conoceremos la longitud de la cadena a priori. Esta función nos permite obtenerla.

`strlen(cadena)`	`mb_strlen(cadena)`

Sencillo ¿verdad? Hagamos unas pruebas.

```php
<?php
  $cadena1 = "maurodosantos@pernambuco.com ";
  echo "Tiene " . strlen($email) . " letras.<br>";
  $cadena2 = "HólaÑ ";
  echo "Tiene " . strlen($cadena2) . " letras.<br>";
```

Salida:

```
Tiene 29 letras //correcto, hay un espacio en blanco al final
Tiene 7 letras //??????
```

WTF?? A ver, a ver, contar hasta 5 sé, y hay 5 caracteres. Cierto, pero, pero... El strlen "cuenta" bytes, es decir, cuenta el espacio que ocupan los caracteres en memoria. ¿Qué ocurre con el carácter ó y con nuestra querida Ñ de Españita? Que, si bien es un carácter, su representación ocupa 2 bytes. Por tanto tanto ó como la Ñ strlen los cuenta como 2 carácteres.
Solución:
Sencilla, **NO USAR strlen,** usar las cadenas de caracteres multibyte. Esto es muy simple, simplemente hay que poner mb_ delante de cada función, en este caso **mb_strlen ()**.
A partir de ahora, pondré las dos funciones pero **debemos usar cadena multibyte.**

NOTA: Todos los ejemplos del tema son con la versión SIN MULTIBYTE, te propongo que realices los cambios en los apuntes y uses la notación multibyte o mb_.

mb_strpos() y strpos().

En ocasiones nos interesará conocer la posición de la primera aparición de un carácter en una cadena. Si no se encuentra el carácter, devuelve FALSE.

```
strpos(cadena, caracter)
```

Alternativamente podemos indicarle la posición a partir de la cual queremos que empiece a buscar.

```
strpos(cadena, caracter, posición inicial)
```

Lo mismo para **mb_strpos()**, pero con mb_ delante.

mb_substr() substr().

Habrá muchos casos en los que nos interesará extraer un trozo del texto de una cadena. Mediante esta función podemos extraer el fragmento ubicado entre las posiciones indicadas:

```
substr(cadena, posición inicial, longitud)
```

Lo mismo para **mb_substr()**, pero con mb_ delante.

Ejemplo 3.1.7. El funcionamiento de esta función se explicará con un ejemplo típico, la comprobación de que un mail es correcto y la extracción de datos del mismo.

```php
<?php
  $email = "maurodosantos@pernambuco.com ";
  echo "Email a analizar: '$email'<br>"; echo "<br>";
  echo "Tiene " . strlen($email) . " letras.<br>";
  // Indica la posición del caracter "@" o FALSE si no está
  $posicion_arroba = strpos($email, "@");
  // Busca la aparición de un punto (.) partir de la arroba
  $posicion_punto = strpos($email, ".", $posicion_arroba);
  if ($posicion_arroba && $posicion_punto) {
    echo "Es una dirección de email válida<br>";
    $usuario = substr($email, 0, $posicion_arroba);
    $dominio = substr($email, $posicion_arroba + 1);
    echo "El nombre de usuario es: $usuario<br>";
    echo "El dominio es: $dominio<br>";
  } else {
    echo "No es una dirección de email válida<br>";
    if (!$posicion_arroba) {
              echo "Le falta el caracter arroba<br>";
    }
    if (!$posicion_punto) {
      echo "El dominio no es válido<br>";
    }
  }
?>
```

trim().

Elimina los espacios en blanco al principio y final de una cadena. En realidad elimina los caracteres definidos en la máscara. Por defecto: espacios, tabuladores, salto de línea y retorno de carro..

```
trim(cadena, [mascara])
```

Algunas alternativas a **trim**, son **rtrim** y **ltrim**. La diferencias radica en que **trim** quita los espacios al principio y al final de la cadena, **rtrim**, sólo al final y **ltrim** sólo al principio de la cadena.

Por supuesto, si no lo digo, esta y otras funciones es que no tiene versión multibye. Además, en este caso no tiene sentido que la tenga, puesto que elimina caracteres, no depende del número de los mismos.

str_replace().

En un texto, reemplaza las apariciones de una cadena por otra.

```
str_replace(cadena a buscar, cadena reemplazar, cadena original)
```

Esta función no tiene equivalente directo en las cadenas multibye, mb_replace NO EXISTE tal cual.
Lo más parecido es **mb_ereg_replace**, lo que pasa es que **mb_ereg_replace**, es una función que permite búsquedas mediante expresiones regulares, es decir es MUCHO MÁS POTENTE y compleja, y como las expresiones regulares, se estudiarán en temas posteriores, aparcamos el análisis de dicha función para más adelante.

Ejemplo 3.1.8. En este ejemplo se supone que podemos recibir emails con errores, y los intentamos arreglar. El primer error que eliminamos es la aparición de espacios en blanco al principio y al final de la dirección. El segundo cuando los usuarios escriben el dominio *pernambuco.es* cuando el correcto es *pernambuco.com*.

```php
<?php
  $email = " maurodosantos@pernambuco.es ";
  echo "Dirección recibida: '$email'.<br>";
  // Eliminamos los espacios en blanco
  $email = trim($email);
  // Reemplazamos pernambuco.es por pernambuco.com
  $email = str_replace("pernambuco.es", "pernambuco.com", $email);
  echo "Dirección corregida: '$email'.";
?>
```

strtoupper(), strtolower() y mb_strtoupper(), mb_strtolower().

La función *strtoupper()* convierte los textos a mayúsculas, y *strtolower()* a minúsculas. Por supuesto es lo mismo con con mb_strtoupper y mb_strtolower.

```
strtoupper(cadena)          mb_strtoupper(cadena)
strtolower(cadena)          mb_strtolower(cadena)
```

Ejemplo 3.1.9. En este ejemplo vamos a pasar la dirección de correo a minúsculas.

```php
<?php
  $email = " MAUROSOSANTOS@PERNAMBUCO.ES ";
  echo "Dirección recibida: '$email'.<br>";

  // Convertimos a minúsculas
  $email = strtolower($email);
  echo "Dirección corregida: '$email'.";
?>
```

Ejemplo 3.1.10. Un uso muy habitual de estas funciones es para comparar cadenas.

Para *PHP* la cadena "MADRID" es diferente a la cadena "Madrid", pero mediante estas funciones se pueden comparar como iguales.

```php
<?php
```

```php
  echo "Comparación de cadenas directamente: ";
  if ("MADRID" == "Madrid") {
    echo "MADRID es igual a Madrid<br>";
  } else {
    echo "MADRID no es igual a Madrid<br>";
  }
  echo "<br>";

  echo "Antes de comparar pasamos ambas a minúsculas: ";
  if (strtolower("MADRID") == strtolower("Madrid")) {
    echo "MADRID es igual a Madrid<br>";
  } else {
    echo "MADRID no es igual a Madrid<br>";
  }
?>
```

number_format().

Esta función es útil cuando hay que mostrar datos de tipo double, es decir, números con decimales. Permite especificar el número de decimales que queremos que se muestren, en lugar de los diez decimales que se muestran habitualmente.

```
number_format(número, decimales, [sep decimal], [sep de miles])
```

Ejemplo 3.1.11. Esta función ya apareció en una práctica del módulo anterior.

```php
<?php
  $precio_kg = 1.29;
  $peso = 2.17;
  $a_pagar = $precio_kg * $peso;
  // Saca el dato $a_pagar ocn 2 decimales
  echo "A pagar " . number_format($a_pagar, 2) . " euros.";
?>
```

Algunas más**. Ninguna tiene versión multibye.**

- **explode(delimitador, string)**. Convierte en array la cadena mediante el delimitador.
- **chop(cadena) o rtrim(cadena)**. Elimina los saltos de línea y los espacios finales de una cadena.
- **ucfirst(cadena)**. Convierte el primer caracter de una cadena a mayúsculas.
- **ucwords(cadena)**. Convierte a mayúsculas el primer caracter de cada palabra de una cadena.
- **md5(cadena)**. Calcula el hash md5 de un string.
- **nl2br(cadena)**. Inserta saltos de línea HTML antes de todas las nuevas líneas de un string.

Ejemplo completo

```
<html>
  <head>
     <title>Ejemplo de cadenas en PHP 7</title>
  </head>
  <body>
  <?php
     // Strlen()
                        echo    "<strong>Ejemplo    de    Strlen()</strong>
<br>".strlen("12345")."<br>";
     echo "<hr>";
```

```
      // Explode()
      $pieza  = "una-dos-tres-cuatro-cinco";
      $piezas = explode("-", $pieza);

      echo "<strong>Ejemplo de Explode()</strong> <br>";

      foreach($piezas as $individuales)
      {
         echo $individuales."<br>";
      }
      echo "<hr>";

      // Substr()
      $cadenaSubstr = "Hola mundo. Esta es una cadena a evaluar.";
      echo "<strong>Ejemplo de Substr()</strong> <br>";
      $cadenaSubstr = substr($cadenaSubstr, 4, 10);
      echo $cadenaSubstr."<br>";
      echo "<hr>";

      // Chop()
      echo "<strong>Ejemplo de Chop()</strong> <br>";
      $cadenaChop = "Hola mundo     ";
      echo "<pre>";
      echo chop($cadenaChop);
      echo "</pre >";
      echo "<hr>";

      // Strpos()
      echo "<strong>Ejemplo de Strpos()</strong> <br>";
      $cadenaStrpos = 'Hola mundo. Esta es una cadena a evaluar.';
      $encontrar   = 'mundo';
      $pos = strpos($cadenaStrpos, $encontrar);

      if ($pos === false)
      {
          echo "Ops! la cadena <i>$encontrar</i> no fue encontrada en la
cadena <strong>$cadenaStrpos</strong>.";
      }
      else
      {
           echo "La cadena <i>$encontrar</i> fue encontrada en la cadena
<strong>$cadenaStrpos</strong>     y     existe     en     la     posición
<strong>$pos</strong>.";
      }
      echo "<hr>";

      // Str_replace()
      echo "<strong>Ejemplo de Str_replace()</strong> <br>";

      $vocales = array("a", "e", "i", "o", "A", "E", "I", "O");
      $cadenSTRreplace = "Hola mundo. Esta es una cadena a evaluar.";
      $reemplazador = array("4", "3", "1", "0", "4", "3", "1", "0");

            $cadenSTRreplace  =  str_replace($vocales  ,  $reemplazador,
$cadenSTRreplace);

      echo $cadenSTRreplace."<br>";
      echo "<hr>";

      // Ucfirst()
      echo "<strong>Ejemplo de Ucfirst()</strong> <br>";
```

```
    $cadenaUCfirst = 'hola mundo';
    $cadenaUCfirst = ucfirst($cadenaUCfirst);

    echo $cadenaUCfirst."<br>";
    echo "<hr>";

    // Ucwords()
    echo "<strong>Ejemplo de Ucwords()</strong> <br>";

    $cadenaUcwords = 'hola mundo';
    $cadenaUcwords = ucwords($cadenaUcwords);

    echo $cadenaUcwords."<br>";
    echo "<hr>";

    // Strtolower()
    echo "<strong>Ejemplo de Strtolower()</strong> <br>";

    $cadenaStrtolower = 'HOLA MUNDO';
    $cadenaStrtolower = strtolower($cadenaStrtolower);

    echo $cadenaStrtolower."<br>";
    echo "<hr>";

    // Strtoupper()
    echo "<strong>Ejemplo de Strtoupper()</strong> <br>";

    $cadenaStrtoupper = 'hola mundo';
    $cadenaStrtoupper = strtoupper($cadenaStrtoupper);

    echo $cadenaStrtoupper."<br>";
    echo "<hr>";

    // Trim()
    echo "<strong>Ejemplo de Trim()</strong> <br>";
    $cadenaTrim = "    Hola mundo    ";
    echo "<pre>";
    echo trim($cadenaTrim);
    echo "</pre >";
    echo "<hr>";

    // Md5()
    echo "<strong>Ejemplo de md5()</strong> <br>";
    $cadenaMD5 = "Cadena cualquiera";
    $cadenaMD5 = md5($cadenaMD5);
    echo $cadenaMD5."<br>";
    echo "<hr>";

    // Nl2br()
    echo "<strong>Ejemplo de Nl2br()</strong> <br>";
    $cadenaNl2br = "Hola\nmundo\nHTML";
    echo "<pre>";

    $cadenaNl2br = nl2br($cadenaNl2br);
    echo $cadenaNl2br;

    echo "</pre >";
    echo "<hr>";

    ?>

  </body>
  </html>
```

Arrays.

Hasta ahora hemos trabajado en nuestros programas con una pequeña cantidad de datos, y sin que la cantidad de los mismos variara a lo largo del programa. Sin embargo, la mayoría de los programas útiles requieren tratar gran cantidad de datos. En este caso sería muy incómodo tener una variable para cada dato.

Afortunadamente, cuando se tratan muchos datos, éstos suelen ser de unos tipos muy similares entre sí. Para tratarlos en grupo, *PHP* ofrece tipos de datos compuestos. Es decir, tipos de datos que permiten almacenar varios datos en una misma variable.

El tipo de datos compuesto más sencillo es el *array*. Es una estructura muy potente, flexible y de uso muy intuitivo. Un array está compuesto por varios *elementos*. Cada *elemento* almacena un *valor* diferente. Y para poder localizar un *elemento* disponemos del *índice* o posición, que podría interpretarse como la dirección del dato en cuestión.

Arrays indexados numéricamente.

El tipo más sencillo de arrays son los indexados numéricamente, en los que el índice de cada elemento corresponde con su posición. Hay que tener en cuenta que, al igual que sucedía con las cadenas, el primer elemento de un array tiene *índice* 0, y no 1 como cabría esperar. De igual modo, el último elemento corresponde al *indice* longitud-1.



Inicializar un array.

Los arrays se suelen almacenar en variables, como cualquier otro tipo de datos.

La forma más sencilla de crear un array es asignando los valores de sus elementos a la vez que se crea. Para ello primero declaramos el nombre de la variable y usamos la función *array()*, a la que le pasamos como parámetros un grupo de valores separados por comas.

Ejemplo 3.2.1. Creación de un array por asignación directa mediante la función *array()*:

```
<?php
  $edades = array(28, 43, 32, 55);
  $formas = array("triángulo", "cuadrado", "círculo");
?>
```

La segunda forma de rellenar un array es añadiéndole elementos al final del array. Para añadir un elemento a un array se usa su identificador seguido de corchetes "[ ]" sin índice, y se le asigna un valor.

Ejemplo 3.3.2. Aquí se puede ver como se crea un array mediante la adición de elementos.

```
<?php
  $paises[] = "Italia"; //Añade el elemento con índice 0
  $paises[] = "Francia"; //Añade el elemento con índice 1
  $paises[] = "Portugal"; //Añade el elemento con índice 2
```

```
?>
```

Ejemplo 3.3.3. Se pueden combinar ambas formas, primero declarar un array directamente y cuando sea necesario ir añadiendo elementos.

```
<?php
  $frutas = array("melón", "sandía", "naranja");
  $frutas[] = "manzana";
  $frutas[] = "melocoton";
?>
```

Existe una tercera forma de definir los arrays y es con array vacío y posteriormente se van rellenando. Además, a partir de php 5.4 se puede utilizar la notación []

Ejemplo 3.3.3.b. Aquí podéis ver la dos formas de definir un array vacío y posteriormente ir añadiendo elementos.

```
<?php
  $frutas = array();
  $frutas[] = "manzana";
  $frutas[] = "melocoton";

  $frutas2 = []; // Esta forma es LA MÁS COMUN a día de hoy
  $frutas2[] = "manzana";
  $frutas2[] = "melocoton";
?>
```

Acceder a un elemento de un array.

Necesitaremos acceder a los elementos de un array con el fin de asignarles valores o de leer su contenido. Esto es muy sencillo, basta poner el índice del elemento al que queremos acceder entre corchetes "[...]".

Ejemplo 3.3.4. Aquí se leen los valores de los elementos de un array para sacarlos por pantalla. Luego se asigna un nuevo valor al primer elemento (recordamos, con índice 0).

```
<?php
  $frutas = array("melón", "sandía", "naranja");
  echo "La primera fruta de la lista es $frutas[0].<br>";
  echo "La segunda fruta de la lista es $frutas[1].<br>";
  echo "La tercera fruta de la lista es $frutas[2].<br>";
  $frutas[0] = "limón"; //Asigna un nuevo valor al elemento 1
  echo "La primera fruta de la lista es $frutas[0].";
?>
```

Recorrer un array.

Los arrays se suelen utilizar para almacenar listas de valores. Por ello, una de las acciones más habituales que se hacen con ellos es recorrerlos de principio a fin para leer o modificar uno o varios de ellos.

Las estructuras iterativas (bucles) que se vieron con anterioridad son el mecanismo más apropiado para recorrer los arrays.

Para hacer dicho recorrido únicamente debemos limitar el número de iteraciones al número de elementos del array (longitud), que podemos averiguar mediante la función *count()*.

```
count(array)
```

Ejemplo 3.3.5. La función *count()* nos indica cuantos elementos contiene un array.

```
<?php
  $frutas = array("melón", "sandía", "naranja");
  echo "La lista contiene " . count($frutas) . " frutas.";
?>
```

Ejemplo 3.3.6. Recorriendo un array para mostrarlo por pantalla.

```
<?php
  $frutas = array("melón", "sandía", "naranja");
  for ($i = 0; $i < count($frutas); $i++) {
   echo "Elemento $i: $frutas[$i]<br>";
  }
```

Ejemplo 3.3.7. Recorriendo un array para modificar todos sus elementos. Aquí se usan dos arrays que están relacionados por sus posiciones.

```
<?php
  $nombres = array("Juan", "Inés", "Andrea", "Roberto");
  $edades = array(33, 28, 45, 52);
  for ($i = 0; $i < count($edades); $i++) {
     echo "$nombres[$i] tiene $edades[$i]
                  años.<br>";
  }
  echo "<br>";
  for ($i = 0; $i < count($edades); $i++) {
    $edades[$i]++;
  }
  echo "Ha pasado un año...<br>";
  for ($i = 0; $i < count($edades); $i++) {
    echo "$nombres[$i] tiene $edades[$i] años. <br>";
  }?>
```

Ejemplo 3.3.8. Recorriendo un array para buscar un elemento. Se vuelve a hacer uso de dos arrays relacionados por las posiciones de sus índices.

En este código se da por supuesto que se va a encontrar el valor buscado. Si no se encontrara el elemento buscado, el programa no daría respuesta alguna.

```
<?php
  $nombres = array("Juan", "Inés", "Andrea", "Roberto");
  $edades = array(33, 28, 45, 52);
  echo "¿Cuantos años tiene Andrea?<br>";
  for ($i = 0; $i < count($nombres); $i++) {
    if ($nombres[$i] == "Andrea") {
   echo "$edades[$i] años";
```

```
  }
}?>
```

Ordenar un array.

La función *sort()* ordena los elementos de un array. Si el array está formado por cadenas de texto considera menores las minúsculas que las mayúsculas.

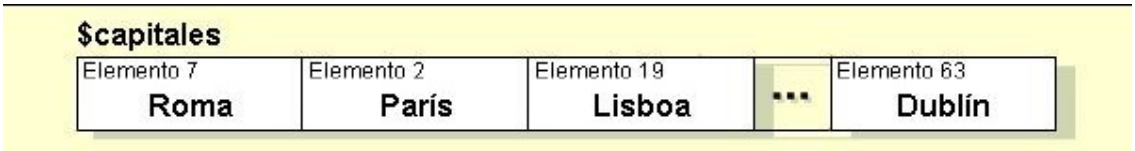
```
sort(array)
```

Ejemplo 3.3.9. En esta ejemplo se ordena una lista de nombres, primero por el procedimiento normal. Luego se muestra la forma de ordenar el mismo array sin hacer distinción entre mayúsculas y minúsculas.

```
<?php
  $astros = array("planeta", "cometa", "Venus", "Jupiter");
  echo "Ordenación distinguiendo mayúsculas y minúsculas:<br>";
  sort($astros);
  for ($i = 0; $i < count($astros); $i++) {
      echo "$astros[$i]<br>";
  }
  echo "<br>";
  echo "Ordenación sin distinguir mayúsculas y minúsculas:<br>";
  $astros_minusculas = array_map("strtolower", $astros);
  array_multisort($astros_minusculas, SORT_ASC, SORT_STRING,
  $astros);
  for ($i = 0; $i < count($astros); $i++) {
      echo "$astros[$i]<br>";
  }
   ?>
```

Arrays no secuenciales.

Hasta ahora lo arrays que hemos visto tenían como índices la sucesión 0 (1er elemento), 1 (2o elemento), 2 (3er elemento), y así sucesivamente.



En *PHP* los índices de un array no tienen por qué ser consecutivos, pueden incluso estar desordenados.

El índice de los arrays asociativos no tiene por qué ser necesariamente un entero, puede ser también un número decimal o una cadena. Este tipo de arrays es más difícil de usar y, aunque permite una mayor flexibilidad, puede ser fuente de muchos errores difíciles de detectar si no se usa con cuidado.

Arrays asociativos.

Éstos son un caso específico de arrays no secuenciales, en los que el índice es una cadena de texto. Pueden resultar útiles para guardar listas, en las que se asocia un valor a una palabra clave.



La creación de este tipo de arrays se puede hacer de dos formas. La primera es mediante la función *array()* de forma parecida a como lo hacíamos antes, solo que ahora deberemos especificar un valor para el índice.

Ejemplo 3.2.10. Cuando se declaran explícitamente los índices se escribe el valor del índice seguido por "=>" y el valor del elemento que contiene.

```
<?php
  $capitales = array("Italia" => "Roma",
                     "Francia"   =>   "Paris",
                     "Portugal"            =>
                     "Lisboa");
  echo "La capital de Francia es {$capitales["Francia"]}";
?>
```

Como se puede observar en el ejemplo, si queremos hacer referencia a un elemento de un array indexado por una cadena dentro de un texto entre comillas, lo tendremos que encerrar entre llaves "{ }". En caso contrario, se produciría un error al interpretar la comilla de apertura del índice del array como si se tratase de la comilla final de la cadena.

Ejemplo 3.2.11. En la siguiente sentencia *PHP* considera como texto la parte en verde y como código la parte en rojo. Como no puede entender la palabra Francia dará un error.

```
  echo "La capital de Francia es $capitales["Francia"]";
```

La segunda forma de crear un array asociativo es añadiendo elementos al array, y asignándoles de forma explícita cual es su índice.

Ejemplo 3.2.12. Veamos esta segunda forma de inicializar arrays declarando explícitamente sus índices.

```
<?php
  $alturas["Aneto"] = 3404;
  $alturas["Teide"] = 3718;
  $alturas["Mulhacen"] = 3748;
  echo "El Aneto mide {$alturas["Aneto"]} metros"; ?>
```

Hay que tener cuidado con este sistema de creación, pues se usa indistintamente para crear un elemento y para acceder a su valor.

Ejemplo 3.2.13. En este ejemplo primero se relaciona un array con otro a través de índices de tipo cadena. Los valores del primer array sirven a su vez como índices del segundo.

```
<?php
  $nombres = array("Eva", "Antonio", "Felipe", "Rosa");
  $edades = array("Eva" => 28,
                  "Antonio" => 43,
                  "Felipe" => 32,
                  "Rosa" => 55);
  for ($i = 0; $i < count($nombres); $i++) {
```

```
  echo "$nombres[$i] tiene {$edades[$nombres[$i]]} años.<br>";
}?>
```

Aún así, al igual que ocurría con los arrays no secuenciales, su uso es desaconsejable si aún no se tiene cierto dominio sobre el lenguaje *PHP*.

Arrays no homogéneos.

PHP establece muy pocas limitaciones a las estructuras de los arrays. Aunque hasta el momento todos los arrays que hemos visto, los hemos usado para manipular datos del mismo tipo (arrays de enteros, arrays de cadenas, etc) esta restricción no existe realmente en *PHP*.

PHP permite mezclar en un array valores de diferentes tipos. Incluso permite que los índices de los elementos de un mismo array sean de diferentes tipos.

Pero una vez más, ésta es una estrategia peligrosa a seguir, y a la que en la mayoría de los casos no será necesario recurrir.

Arrays multidimensionales.

El contenido de un elemento de un array puede, a su vez, ser un array. De esta forma se pueden construir arrays multidimensionales.

Ejemplo 3.2.14. Se muestra como almacenar una matriz, o array de dos dimensiones, que contiene una sopa de letras generada aleatoriamente, mediante la función *chr()*, que devuelve un carácter dado su número ascii.

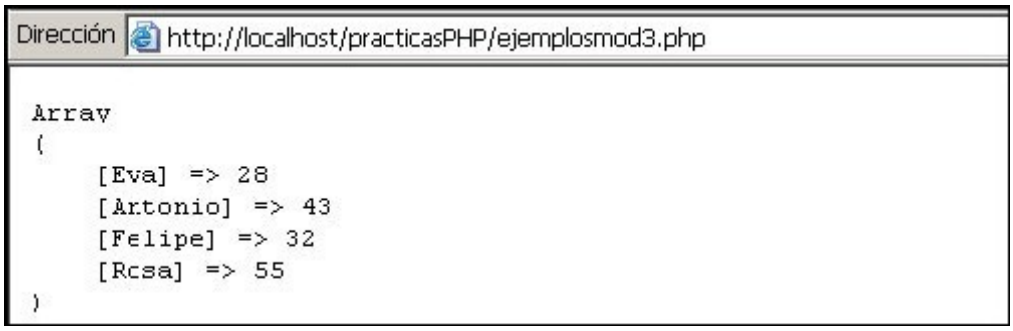
```
<font face="Courier New">
<?php
  DEFINE("ALTO",10);
  DEFINE("ANCHO",20);
  $sopa_letras = array();
  for ($i = 0; $i < ALTO; $i++) {
    for ($j = 0; $j <ANCHO; $j++) {
        $sopa_letras[$i][$j] = chr(rand(65, 90));
    }
  }
  echo "SOPA DE LETRAS<br>”;
  for ($i = 0; $i < ALTO; $i++) {
    for ($j = 0; $j <ANCHO; $j++) {
       echo $sopa_letras[$i][$j];
    }
        echo "<br>";
  }?></font>
```

Mostrar el contenido de un array.

Ejemplo 3.2.15. *PHP* dispone de la función *print_r()* a la que se le pasa como argumento un array, e imprime por pantalla su contenido. Para poderlo ver bien se usa la etiqueta HTML “<pre>… </pre>” que sirve para mostrar el texto y los saltos de línea tal cual se ha escrito.

```
<pre>
<?php
  $edades = array("Eva" => 28,
```

```
                "Antonio" => 43,
                "Felipe" => 32,
                "Rosa" => 55);
  print_r($edades);
?>
</pre>
```



Desbordamiento de arrays.

Si se intenta acceder a un elemento que no existe en un array *PHP* nos devolverá la cadena vacía (""). Esta es una causa de error bastante habitual, especialmente si después vamos a hacer operaciones con el dato extraído.

Ejemplo 3.2.16. Aquí se accede al quinto elemento del array que, en realidad, no existe.

```
<?php
  $formas = array("triángulo", "cuadrado", "círculo");
  echo "El cuarto valor es: '" . $formas[4] . "'";
```

Sentencia foreach.

A partir de la versión 4 de *PHP* se incluyó la sentencia de control *foreach* que permite recorrer arrays de forma cómoda. Hay dos sintaxis disponibles. La primera es un bucle que recorre todos los elementos del array. En cada ciclo la variable *$valor* toma el valor del elemento actual.

```
foreach(array as $valor) {
  // bloque de código
}
```

La segunda sintaxis es similar, solo que además de tomar la variable *$valor* el valor del elemento actual, la variable *$key* tomará el valor del índice (*key* o clave).

```
foreach(array as $key => $valor) {
  // bloque de código
}
```

Resulta especialmente útil cuando se usa en conjunción con arrays asociativos.

PHP 8. Nueva Función: array_is_list

La función array_is_list() en PHP 8.1 devuelve verdadero si un array es una lista ordenada y secuencial de elementos, comenzando por el índice 0, y falso en caso contrario.

```
<?php
$array1 = [1, 2, 3, 4, 5];
array_is_list($array1); // true

$array2 = [0 => 1, 1 => 2, 2 => 3, 3 => 4, 4 => 5];
array_is_list($array2); // false

$array3 = [1 => 1, 2 => 2, 3 => 3, 4 => 4, 5 => 5];
array_is_list($array3); // false

$array4 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
array_is_list($array4); // true

$array5 = ['a' => 'apple', 'b' => 'banana', 'c' => 'cherry'];
array_is_list($array5); // false
```

Comprobación y eliminación de posiciones de Arrays.

Es una situación muy habitual (sobretodo en arrays asociativo) tener que comprobar si una posición de un array existe, o en ocasiones eliminar la posición de un array. Las funciones que realizan esto, sólo sirven para arrays, pero sí son muy habituales en un array.

isset()

La función isset comprueba si una variable está definida o no en el script de PHP que se está ejecutando. Determina si una variable está definida y no es NULL

Otras funciones similares son:

- PHP is_null(): Para comprobar si una variable es NULL.
- PHP empty(): Para saber si una variable está vacía.

Sintaxis:

isset ($var1, $var2, $var3...)

- La función devuelve **TRUE** o **FALSE**.
- **var1 (Obligatorio)**: Variable que queremos comprobar si está definida.
- **var 2, var3... (Opcionales)**: Variables adicionales a comprobar.

Ejemplos de PHP isset()

```
//Ejemplo con variable definida
$variable = 'valor';
if (isset($variable))
{
   echo "Variable definida!!!";
}else
{
  echo "Variable NO definida!!!";
}

//Ejemplo con variables definidas
$variable = 'valor';
```

```
$variable2 = 'valor2';
if (isset($variable,$variable2))
{
   echo "Variables definidas!!!";
}else
{
   echo "Variables NO definidas!!!";
}

//Ejemplo con variables NO definidas
$variable = 'valor';
$variable2 = 'valor2';

if (isset($variable,$variable2, $variable3))
{
   echo "Variables definidas!!!";
}else
{
echo "Variables NO definidas!!!";
}
```

Ejemplos de PHP isset() con arrays

```
$jugadores = array ('portero' => 'Casillas', 'Defensa' => 'Pepe');
if (isset($jugadores['portero']))
{
        echo "Variable de array definida!!!";
}else
{
        echo "Variable de array NO definida!!!";
}

$jugadores = array ('portero' => 'Casillas', 'Defensa' => 'Pepe');
if (isset($jugadores['delantero']))
{
        echo "Variable de array definida!!!";
}else
{
        echo "Variable de array NO definida!!!";
}
```

El uso más habitual de isset es con los array superglobales $_GET, $_POST y $_REQUEST.

unset()

La función unset elimina la variable siempre y cuando esté definida. Elimina cualquier tipo de variable, array u objetos.

Sintaxis:

unset ($var1, $var2, $var3...)

Ejemplos de PHP unset()

```
$variable1 = "valor1";
```

```
$variable2 = "valor2";
$variable3 = "valor3";
var_dump ($variable1, $variable2, $variable3);

unset($variable1, $variable2, $variable3);
var_dump ($variable1, $variable2, $variable3);
//muestro el array numerico original
$array1 = array(1,2,3,4,5,6);
var_dump ($array1);

//elimino el tercer elemento del array y muestro el array
unset($array1[2]);
var_dump ($array1);

//elimino el primer y segundo elemento del array y muestro el array
unset($array1[0],$array1[1]);
var_dump ($array1);
//Declaro el array asociativo y muestro su valor
$array2 = array(uno=>1, dos=>2, tres=>3, cuatro=>4, cinco=>5, seis=>6);
var_dump ($array2);

//Elimino el elemento con clave = dos y muestro el array
unset($array2['dos']);
var_dump ($array2);

//Elimino los elementos con clave = uno y cinco, despues muestro el array
unset($array2['uno'],$array2['cinco']);
var_dump ($array2);
```

Funciones.

PHP es un lenguaje estructurado, y como tal dispone de funciones. Las funciones no son más que fragmentos de código, que podrían verse como pequeños programas, y que pueden ejecutarse posteriormente en resto del código. Gracias a esto permiten:

- Reutilizar código que se usa frecuentemente.
- Estructurar lógicamente el código de la aplicación para que sea más comprensible.
- Separar el código en diferentes ficheros para poder compartirlo en diferentes páginas *PHP*.

Funciones definidas por el usuario.

Declaración de una función.

Antes de poder utilizar una función ésta debe ser escrita. Por lo tanto, el lugar en el que se suelen situar las funciones es al comienzo del archivo, de tal forma que estén disponibles a partir de ese momento. Situarlas al comienzo también ayuda a una estructuración más lógica del código.

Las funciones se declaran, al igual que las variables, con un nombre. Éste irá precedido de la palabra *function*, unos paréntesis (para albergar los datos de entrada, llamados argumentos o parámetros) y unas llaves que incluyen el cuerpo de la función. Dentro del cuerpo de la función podemos declarar variables, llamadas a otras funciones y demás sentencias.

```
function nombre_funcion (arg1, arg2, arg3, ...) {
```

```
    // bloque de código
    return valor; //Opcional
}
```

Llamada a una función.

Para utilizar una función se escribe su nombre, seguido de paréntesis y dentro de estos se escriben los datos que se quieren pasar a la función (o variables que contienen dichos datos).

```
nombre_funcion (arg1, arg2, arg3, ...);
```

Si la función devuelve un dato, éste se puede asignar a una variable o usar directamente como parte de una expresión.

Ejemplo 3.3.1. A continuación se muestra una función muy sencilla, que no tiene argumentos.

```
<?php
function hola_mundo()
 {
      echo "¡Hola Mundo!";
 }

 hola_mundo();
 ?>
```

Devolución de un valor.

Las funciones son mucho más útiles si pueden devolver un dato. Para ello usan la sentencia *return*. Una vez que se llega a esta instrucción no se ejecuta el código que se pueda encontrar a continuación.

Ejemplo 3.3.2. La siguiente función devuelve aleatoriamente un día de la semana.

```
<?php
  function dia_semana() {
    $semana = array("lunes", "martes", "miércoles",
                    "jueves", "viernes", "sabado", "domingo");
    $dia = $semana[rand(0, 6)];
    return $dia;
  }
  $dia_cine = dia_semana();
  echo "El próximo $dia_cine voy al cine.";
?>
```

Argumentos.

Aún así, estas funciones que hemos visto son muy sencillas y no permiten hacer gran cosa. Lo más habitual es que a las funciones se les pasen datos, para que luego operen con éstos, y que al terminar la función devuelva el resultado. A los datos que recibe una función se les llama argumentos o parámetros.

En la declaración de la función, tras la palabra clave *function*, va el nombre de la función seguido por una lista de argumentos entre paréntesis y separados por comas. Dentro del cuerpo de la función estos datos se pueden utilizar como una variable cualquiera.

Ejemplo 3.3.3. Función sencilla que recibe un parámetro y devuelve un resultado. Luego es llamada dentro de un bucle, para mostrar el cuadrado de los números del 1 al 10.

```php
<?php
function cuadrado($numero) {
  return $numero * $numero;
  }
  for ($i = 1; $i <= 10; $i++) {
    echo "$i al cuadrado es igual a " . cuadrado($i) . ".<br>";
  }
?>
```

Paso de argumentos por valor.

Cuando se pasan valores a las funciones, podemos pasarlos de 2 formas distintas: por valor y por referencia. El comportamiento predefinido en *PHP* se conoce como paso por valor.

En el paso de argumentos por valor, la variable que recibe el valor hace una copia del mismo, convirtiéndose en una nueva variable, y por tanto actuando a partir de ese momento como una variable totalmente independiente.

El aspecto práctico más importante consiste en que si cambiamos en algún momento el valor del argumento dentro de la función (copia), no se ve modificado fuera de ella (original).

Paso de argumentos por referencia.

En ocasiones es preferible no hacer la copia del dato que se pasa, sino trabajar directamente sobre el dato original.

Esto, por ejemplo, puede interesar cuando se trabaje con arrays, ya que el copiado de muchos datos puede perjudicar al tiempo de ejecución del programa, o más comúnmente cuando se quiere alterar el valor de la variable para obtener algún efecto.

A esto se le llama paso de parámetros por referencia. Para indicar que un parámetro se pasa por referencia se antepone el símbolo ampersand, "&", en la declaración de la función.

Una misma función puede emplear argumentos pasados por valor y por referencia, en una misma declaración.

Ejemplo 3.3.4. Ejecutando el siguiente código podemos observar los diferentes resultados obtenidos entre utilizar la misma función empleando un paso de parámetros por valor o por referencia. Como es de esperar, mediante el paso por referencia la variable exterior se verá afectada por los cambios ocurridos en el interior de la función.

```php
<?php
  function duplicar_por_valor($argumento) {
    $argumento = $argumento * 2;
    echo "Dentro de la función vale $argumento.<br>";
  }

  function duplicar_por_referencia(&$argumento) {
    $argumento = $argumento * 2;
    echo "Dentro de la función vale $argumento.<br>";
  }
```

```
  $numero1 = 5;
  echo "Antes de llamar a la función vale $numero1.<br>";
  duplicar_por_valor($numero1);
  echo "Después de llamar a la función vale $numero1.<br>";
  echo "<br>";
  $numero2 = 7;
  echo "Antes de llamar a la función vale $numero2.<br>";
  duplicar_por_referencia($numero2);
  echo "Después de llamar a la función vale $numero2.<br>";
?>
```

Ejemplo 3.3.5. Aunque el lenguaje no nos permite hacer que una función devuelva dos valores, tenemos la posibilidad de pasar por referencia las variables a devolver y, así, modificar su contenido en el cuerpo de la función.

Aquí se emplea esta estratagema para intercambiar el contenido de dos variables.

```
<?php
  function intercambiar(&$argumento1, &$argumento2) {
    $auxiliar = $argumento1;
    $argumento1 = $argumento2;
    $argumento2 = $auxiliar;
  }

  $numero1 = 5;
  $numero2 = 8;
  echo "Antes: ($numero1, $numero2)<br>";
  intercambiar($numero1, $numero2);
  echo "Despues: ($numero1, $numero2)";?>
```

Alcance de las variables.

Dentro de las funciones también podemos declarar nuevas variables, pero, ¿que pasa si hay una variable dentro de una función que se llama igual que una variable externa? La respuesta es que la variable interna es diferente de la que está fuera, y por lo tanto su valor será totalmente independiente.

A esto se le llama *alcance de una variable*, y tiene las siguientes implicaciones:

- Las variables que se declaran dentro de una función sólo existen en el cuerpo de dicha función.
- Las variables globales, que son aquellas que no se han declarado dentro de ninguna función, son accesibles en cualquier momento, independientemente del punto del código en que nos encontremos.
- Las variables locales, que son las declaradas dentro de una función, tienen preferencia sobre las variables globales.

Ejemplo 3.3.6. Este código muestra el alcance de una variable dentro de una función.

No hay que confundir este ejemplo con el ejemplo 3.3.4. En este caso no hay ningún argumento.

```
<?php
      function
   mi_ciudad() {
   $ciudad = "Madrid";
```

```
    echo "Dentro de la función vale $ciudad.<br>";
  }

  $ciudad = "Barcelona";
  echo "Antes de llamar a la función vale $ciudad.<br>";
  mi_ciudad();
  echo "Después de llamar a la función vale $ciudad.<br>"
?>
```

Ejemplo 3.3.7. Puede darse el caso de que queramos acceder a una variable global dentro del cuerpo de la función. Para conseguirlo le antepondremos la palabra clave *global* a la primera referencia de la variable, con lo que el interprete *PHP* sabe que estamos llamando a la variable externa.

```
<?php
      function
    mi_ciudad() {
    global $ciudad;
    $ciudad = "Madrid";
    echo "Dentro de la función vale $ciudad.<br>";
  }

  $ciudad = "Barcelona";
  echo "Antes de llamar a la función vale $ciudad.<br>";
  mi_ciudad();
  echo "Después de llamar a la función vale $ciudad.<br>"
?>
```

No obstante, es recomendable usar en las funciones nombres de variables diferentes a los de las variables del programa principal.

Tampoco es conveniente usar variables globales dentro de las funciones. Es mejor pasar estas variables como parámetro, ya que mejora notablemente la fiabilidad y claridad del código.

Creación de bibliotecas.

Para conseguir un código lo más claro posible, es deseable que éste sea breve. Una forma de conseguirlo es extraer las funciones que se declaran a un archivo independiente con extensión "*.php*". Una ventaja de esta estrategia es que se pueden hacer accesibles estas funciones a más de una página en *PHP*.

Ejemplo 3.3.8. A continuación se muestra una biblioteca de funciones, guardada en el archivo "*utils.php*", en la que se ha optado por comentarios, para mejorar la legibilidad y mantenibilidad del código.

```
<?php
  // Biblioteca de funciones de usuario: utils.php
  // Devuelve el argumento elevado al cuadrado
  function cuadrado($numero) {
 return $numero * $numero;
  }

  // Devuelve la raíz cuadrada del argumento
  function raiz($numero) {
 return sqrt($numero);
```

```
  }

  // Devuelve verdadero si el número es igual o mayor que cero
  function es_positivo($numero) {
    return ($numero >= 0);
  }
?>
```

Ejemplo 3.3.9. En los ficheros de biblioteca se puede incluir cualquier tipo de código.

También es posible crear y encontrarse archivos de configuración o de constantes predefinidas.

En este caso se guarda en el archivo "*config.php*".

```
<?php
  // Fichero de configuración
  // Archivo config.php
  DEFINE("PI", 3.1416);
  DEFINE("NUMERO_E", 2.7183);
  DEFINE("EULER", 0.5772);
?>
```

Las instrucciones include() y require().

Para tener disponibles las funciones de un fichero externo hay que indicarle al código *PHP* que debe incorporarlas al script actual.

Mediante estas instrucciones se incluye el contenido del fichero importado en el punto exacto en el que realiza la importación.

La diferencia entre las funciones de importación disponibles reside en que *require()* lanza un error fatal en el caso de no encontrar el fichero a importar, mientras que *include()* no lo hace.

Ejemplo 3.3.10. La inclusión de bibliotecas se suele poner al principio del código para que estén disponibles en el resto del código.

```
<?php
  include("config.php");
  include("utils.php");
  $radio = 5;
  $circunferencia = 2 * $radio * PI;
  $area = PI * cuadrado($radio);
  echo "Un círculo de rádio $radio tiene circunferencia ";
  echo "$circunferencia y área $area.<br>";
  $area = -8;
  if (es_positivo($area)) {
    $radio = raiz($area / PI);
    echo "Un círculo de área $area tiene un rádio $radio";
  } else {
    echo "No se puede calcular, área negativa.";
  }
?>
```

Hay que tener en cuenta el orden en el que se cargan los archivos externos, especialmente si unos hacen uso del código de otros.

require_once e include_once

Cuando la cantidad de includes es muy grande existe la posibilidad que hayan archivos dependientes o incluso bucles de includes

. Por ejemplo

Fichero a.php
```
include 'b.php';
```

Fichero b.php
```
include 'b.php';
```

Fichero c.php
```
include 'a.php';
```

El fichero A incluye el B y el fichero B incluye el C y el fichero C incluye el A.
Problema: Tenemos un bucle.

Otro ejemplo:

Fichero a.php
```
include 'b.php';
include 'c.php';
```

Fichero b.php
```
include 'c.php';
```

Fichero A incluye B y C
Fichero B incluye C.
Problema: Incluimos dos veces el C. Cargamos dos veces→ Error!!!!

Para evitar la carga duplicada (y los errores que ellos conlleva) y los bucles, podemos utilizar require_once e include_once.

Ambas funcionan igualmente que sus versiones sin **_once** pero al utilizar requier_once o include_once, sólo se carga una vez cada fichero incluido, por tanto si detecta que quiere cargar un fichero anteriormente cargado, no lo hace y no se producen dichos errores.

Otros tipos de funciones.

Hasta ahora hemos visto las funciones que nosotros definimos. Pero a lo largo de muchos ejemplos hemos ido usando funciones como *rand()* o *number_format()* que ya estaban disponibles en el lenguaje. Éstas son funciones del núcleo de *PHP*, y por lo tanto no hace falta hacer un *include()* para poder usarlas.

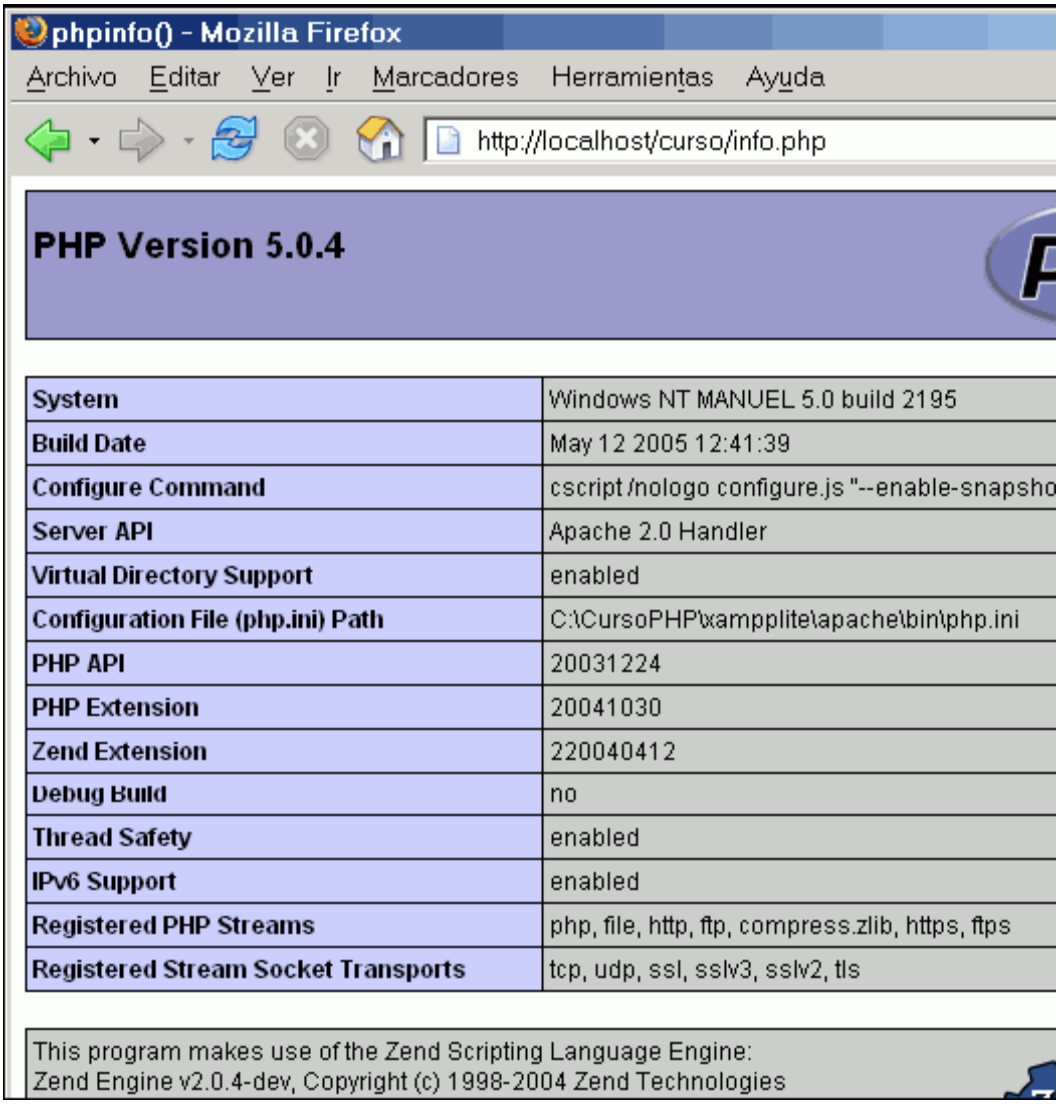
Extensiones de PHP.

Las funciones que incorpora de forma predefinida *PHP* están agrupadas en extensiones.

Ejemplo 3.3.11. Para saber que extensiones están instaladas en nuestro interprete de *PHP* tenemos una opción muy útil, la función *phpinfo()* que genera una página con la información del intérprete que tenemos instalado.

```php
<?php
  phpinfo();
?>
```

Debe aparecernos una pantalla similar a la siguiente:



Referencia sobre funciones.

Toda la referencia sobre funciones se puede encontrar en la página oficial sobre *PHP*: http://es.php.net/manual/es/funcref.php.

Hay una gran multitud de funciones en *PHP* de entre las cuales probablemente usaremos sólo unas pocas. Las extensiones más útiles son:

Funciones matemáticas:
http://es.php.net/manual/es/ref.math.php.

Funciones de cadenas:
http://es.php.net/manual/es/ref.strings.php.

Funciones de mysql:
http://es.php.net/manual/es/ref.mysql.php.

Funciones de fecha y hora:

http://es.php.net/manual/es/ref.datetime.php.

La página dispone también de un buscador que resulta bastante útil.

Funciones. Novedades incorporadas en php

Tipado fuerte en PHP: Descriptivo y Estricto

Como hemos visto hasta ahora, PHP es un lenguaje de programación de tipado débil,vamos que no es necesario definir el tipo de las variables, ya que lo toma automáticamente.

Esto que puede parecer una ventaja, puede conllevar código extra a la hora de manejo de tipos y ser muy cuidadoso con los cambios automáticos y no todos los programadores lo consideran una buena práctica. Por tanto, cuando PHP presentó su nueva versión añadió una serie de características y una de ellas es el chequeo de tipos en el paso de parámetros y devolución de funciones.

A partir de php 7.0, para satisfacer las necesidades de muchos programadores introduzco tipado FUERTE. Si bien existe dos tipos de tipado Fuerte en php. El tipado **fuerte descriptivo** y el tipado **fuerte estricto.**

Con el tipado fuerte, conseguimos hacer las funciones más expresivas al definirlas. Además, si se llaman con datos incorrectos el intérprete se quejará o se generará un error, dependiendo de nuestra configuración. En función del comportamiento que elija (descriptivo ó estricto).

Tipado fuerte descriptivo

En el tipado descriptivo, la diferencia fundamental es que realizará la conversión de tipos automáticamente siempre y cuando sea compatible.

Ahora veamos cómo se define una función en donde se comprueban los tipos de datos, escribiendo de nuevo la función de arriba. Queda así:

```
function primeraLetra (string $frase):string{
      return $frase[0];
      }
```

Ahí ya estamos indicando que recibimos una variable de tipo `string`, y que devolvemos igualmente un `string`. Cuando otro programador lea la definición de la función sabrá exactamente lo que dicha función hace.

Además Incluso si la llamamos con el número `500`, PHP lo convertirá a cadena.

```
echo "La primera letra de 500 es: " . primeraLetra(500);
```

Quedando la salida de la siguiente manera

```
      La primera letra de 500 es: 5
```

¿Pero que pasaría en el caso de que en vez de una función que recibe un int recibiera una cadena de texto?

```
define ('PI', 3.141591);
function calculaAreaCirculo (int $radio):float{
      return PI* pow($radio, 2);
      }
//Esto funciona correctamente
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo(5);
// Esto funciona pues realiza la conversión
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo('5');
//Esto falla pues no es un tipo compatible
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo('aa');
```

La última llamada genera error, pues no sólo no es del tipo correcto, sino que además no puede realizar el casteo (“palabro” para conversión de tipos) correctamente.

Por cierto, no únicamente podemos pasar strings. También acepta el tipo de dato `array`, `callable`, `bool`, `float` e `int`. (Sí, hay booleanos en php)

Tipado fuerte Estricto

Aquí lo que hacemos es forzar a php para que funcione en modo estricto de manera que **NO REALICE EL CASTEO AUTOMÁTICO** de tipos.

Ejemplo:

```
declare(strict_types=1);
define ('PI', 3.141591);
function calculaAreaCirculo (int $radio):float{
      return PI* pow($radio, 2);
      }
//Esto funciona correctamente el número es entero
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo(5);
// Evidentemente lo siguiente falla, pues ni si quiera es casteable
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo('aa');
//Esto falla pues aunque es convertible, es de tipo string
echo "Área de circulo de radio: $radio es: " . calculaAreaCirculo('5');
```

En el último caso se produce un error debido a que, aunque podría realizar el casting, le hemos dicho que no lo haga, y '5' no es un datos int.

Lo ideal al respecto de la declaración de tipos estricta hubiera sido **poder declararla a nivel de proyecto**. No obstante, dado que si definimos algo así en el php.ini, afectaría al código de nuestras dependencias, los desarrolladores de php 7 prefirieron ser conservadores y definirlo a nivel de proyecto.

Listas de argumentos de longitud variable ¶

PHP tiene soporte para listas de argumentos de longitud variable en funciones definidas por el usuario. Esto se implementa utilizando el token ... (tres puntos) y utilizando las funciones func_num_args(), func_get_arg(), y func_get_args() en versiones anteriores de php

Si bien esta característica es introduce con anterioridad al 8, la pongo como actualización.

Las listas de argumentos pueden incluir el token ... para denotar que la función acepta un número variable de argumentos. Los argumentos serán pasados a la variable dada como un array, por ejemplo:

Ejemplo. Usando ... para acceder a argumentos variables

```
<?php
function sum(int ...$números) {
    $acc = 0;
    foreach ($números as $n) {
        $acc += $n;
    }
    return $acc;
}
echo sum(1, 2, 3, 4);
?>
```

El resultado del ejemplo sería: 10

También se puede emplear **...** al llamar a funciones para convertir un array o un literal en una lista de argumentos:

Ejemplo. Usar ... para proporcionar argumentos

```
<?php
function add($a, $b) {
    return $a + $b;
}
echo add(...[1, 2])."\n";
$a = [1, 2];
echo add(...$a);
?>
```

El resultado del ejemplo sería: 3

Se puede especificar argumentos posicionales normales antes del token .... En este caso, solamente los argumentos al final que no coincidan con un argumento posicional serán añadidos al array generado por ...

Union Types

En PHP 8 ya podemos utilizar Union Types, una característica muy poderosa de otros lenguajes de programación. La idea es poder utilizar múltiples tipos de datos para argumentos, datos de retorno y propiedades en lugar de poder utilizar únicamente un tipo de dato, veamos un ejemplo.

```
public function foo(Foo|Bar $input): int|float
```

Como puedes ver, ahora en los argumentos de la función aceptamos un dato que puede ser de tipo Foo o de tipo Bar (en realidad esto tipos no existen), además, el retorno de datos puede ser de tipo string o int.

Otro Ejemplo

```
public function (int|float $n1, int|float $n2): int|float {
      if ($n1%$n2==0) return intdiv($n1,$n2);
      else return $n1/$n2;
}
```

Expresiones match, ¿mejora del switch?

El bloque switch en PHP lo solemos utilizar de vez en cuando para comprobar un valor a través de una serie de casuísticas. En PHP aparece la expresión match, las cuales no requieren de un break, pueden ser almacenadas en variables y utilizan la comparación de datos estricta, veamos un ejemplo.

```
<?php
$phpVersion = 8.0;
echo match ($phpVersion) {
    '8.0' => "¡Casí!, no estás a la última!",
    8.0 => "¡Estás a la última!",
};
```

Ejemplo

Antes de php 8

```
<?php
switch (gettype($value)) {
    case 'array':
        echo 'Es array';
    case 'integer':
        echo 'Es entero';
    case 'double':
    case 'float':
        echo 'Es Decimal.';
    case 'string':
        echo 'Es Cadena texto';
    case 'boolean':
        echo ($value ? 'Es bool y true' : 'Es bool y false')
    case 'NULL':
    case 'null':
        echo 'Es Nulo';
```

```
    default:
        echo ('Tipo extraño');
}
```

En php 8

```
echo match(gettype($value)) {
    'array' => 'Es array',
    'integer' => 'Es Entero',
    'double', 'float' => 'Es Decimal',
    'string' => Es Cadena de texto',
    'boolean' => ($value ? 'Es bool y true' : 'Es bool y false'),
    'NULL', 'null' => ‘Es nulo',
    default => 'Tipo extraño',
};
```

Otra novedad de match es que se pueden comparar varios valores:

```
return match($value) {
    'manzana','plátano' => 'fruta',
    'tomate','pepino' => 'hortalizas',
};
```

Otro dato importante, PHP 8 también soporta una coma al final de una lista de parámetros.

Nuevo tipo mixed

Mixed es un nuevo tipo de dato en PHP 8 algo peligroso, digo peligroso porque si se le da mal uso puede convertirse en algo malo. El problema de mixed es que nos permite definir un tipo de dato que puede contener cualquiera de los siguientes tipos: array, bool, callable, int, float, null, object, resource, string.

```
function NombreFuncion (mixed|FooClass $bar): int|mixed {}
```

Si tienes experiencia programando sabrás que si a eso se le da mal uso el resultado será una aplicación que no conoce el tipo de dato que recibe con los consiguientes inconvenientes, veamos un ejemplo rápido de uso.

```
<?php

function Operar (mixed $dato1, mixed $dato2):int|mixed
{
      if  (gettype($dato1)=='array')  return  asort($array);  //devolvemos
array
      else if (gettype($dato1)=='integer'&& gettype($dato2)=='integer') {
            return $dato1+$dato2; //devolvemos int
      }
      else return $dato1/$dato2;    //devolvemos float
}
```

Sintaxis First-class Callable function

En PHP 8.1 es posible obtener una referencia a cualquier función; esto se llama sintaxis First-class Callable. Para entender como funciona veamos un ejemplo anterior a php 8.1.

```
function cubo($n)
{
    return ($n * $n * $n);
}

$a = [1, 2, 3, 4, 5];
$b = array_map('cubo', $a);
print_r($b);
/*
Array
(
    [0] => 1
    [1] => 8
    [2] => 27
    [3] => 64
    [4] => 125
)*/
```

En php 8.1 el código queda de la siguiente manera.

```
function cubo($n)
{
  return ($n * $n * $n);
}

$a = [1, 2, 3, 4, 5];
$b = array_map(cubo(...), $a);
print_r($b);
/*
Array
(
  [0] => 1
  [1] => 8
  [2] => 27
  [3] => 64
  [4] => 125
)*/
```

Como puedes ver, ahora se ha vuelto más legible ya que en lugar de usar una cadena, ahora llamamos, directamente, a la función misma.

Nuevo tipo de retorno never

Una función o método declarado con el tipo never indica que no devolverá un valor y producirá una excepción o finalizará la ejecución del script con una llamada de die(), exit(), trigger_error(), o similar.

```
<?php

function redirect(string $uri): never {
    header('Location: ' . $uri);
    exit();
}

function redirectToLoginPage(): never {
    redirect('/login');
    echo 'Hello'; // <- dead code detected by static analysis
}
```

Formularios.

Hasta este momento todas las páginas que hemos visto a través de ejemplos tenían un defecto, no eran interactivas. Únicamente hacían operaciones en función de los datos de los que ya disponían, pero eran incapaces de recibir datos del navegador. Es decir, de un usuario externo al propio sistema.

Una de las formas más habituales de recibir datos por parte de una página web dinámica es a través de los formularios web. Estrictamente hablando, los formularios no son parte del lenguaje *PHP* sino del lenguaje *HTML*.

Sin embargo, y aunque hay algunas excepciones, los formularios *HTML* suelen ser recibidos por páginas dinámicas encargadas de procesarlos y/o almacenarlos.

Hay que tener en cuenta que además de formularios *HTML* hay otras tecnologías que permiten el envío de datos a páginas web, con Flash u otras páginas web dinámicas.

Formularios HTML.

Los formularios pueden llegar a ser muy complejos, con una gran cantidad de campos de entrada, validación de datos por *Javascript*,campos ocultos, y otros. Pero el objetivo del curso no es explicar su funcionamiento a fondo, así que se hará un repaso muy superficial.

Un formulario está formado por tres tipos de elementos: etiquetas de comienzo y final, campos de entrada de datos y botones para realizar acciones.

Declaración del formulario.

Un formulario se distingue por estar encuadrado entre una etiqueta de comienzo de formulario "*<form ...>*" y una de final de formulario "*</form>*". La etiqueta de comienzo de formulario ha de tener al menos tres atributos.

- *name*: Aunque no es obligatorio, es muy recomendable darle un nombre al formulario para que sea fácil de identificar.
- *method*: Este parámetro puede tomar dos valores según la forma de enviar los datos:
- *get*: los datos se envían codificados en la URL o dirección de la página destino.
- *post*: los datos se envían ocultos dentro del contenido de la petición.
- *action*: indica la página destino que se encargará de procesar los datos que envía el formulario actual.

```
<form name="formulario" method="get" action="procesar.php">
  Campos de entrada de datos
   Botones de acciones
</form>
```

Campos de los formularios.

Una vez declarado el formulario hemos de poner un campo para cada uno de los datos que queremos recibir. Hay 6 tipos de campos:

Texto: se usa para recibir una línea de texto.

```
<input type="text" name="nombre">
```

Selección única: se usa para elegir una opción entre varias. Si se quiere que las opciones sean excluyentes entre sí han de tener el mismo nombre, y diferentes valores.

```
<input type="radio" name="sexo" value="hombre">
<input type="radio" name="sexo" value="mujer">
```

Selección múltiple: se usa para elegir una o más opciones de un grupo.

```
<input type="checkbox" name="cine">
<input type="checkbox" name="musica">
<input type="checkbox" name="lectura">
```

Lista de selección: es un sistema alternativo al campo de selección única para elegir una opción entre varias, esta vez con formato de lista desplegable.

```
<select name="sexo">
  <option>hombre</option>
  <option>mujer</option>
</select>
```

Área de texto: sirve para poder recibir textos largos compuestos por varias líneas.

```
<textarea name="opinion"></textarea>
```

Campo oculto: en ocasiones queremos pasar un dato a la página *PHP* que procesará el formulario, pero no queremos que esté se muestre en un campo visible. Puede ser útil para pasar información entre páginas *PHP* independientes.

```
<input type="hidden" name="referencia">
```

En todos ellos es importante utilizar el atributo *name*, que contendrá el nombre de variable con el cual vamos a recibir los datos. El atributo *value* es opcional e indica el valor predefinido que se les da.

Botones de los formularios.

Todos los formularios han de tener un botón para enviar los datos una vez que se han rellenado.

```
<input type="submit" name="Enviar">
```

También se suele incluir un botón para devolver los campos del formulario a su estado y/o valor inicial (puede que vacíe el campo).

```
<input type="reset" name="Borrar todo">
```

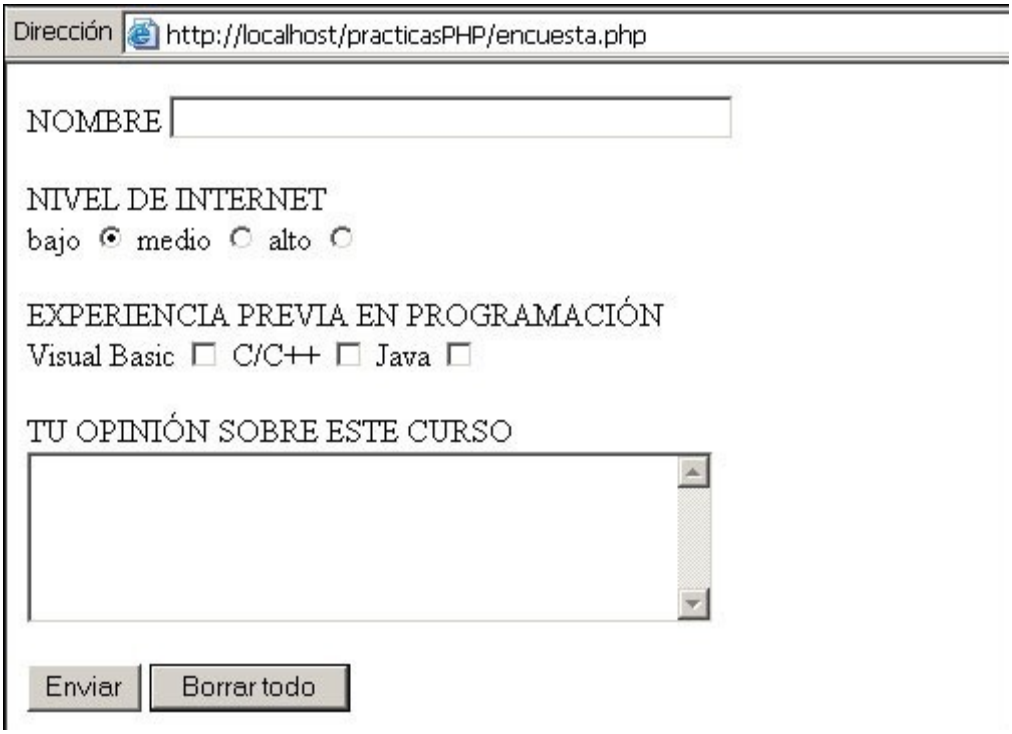
También disponemos de los botones de tipo "*button*" que se emplean para introducir código de la parte cliente, generalmente *Javascript*. Se suelen usar para facilitar y/o verificar parte o la totalidad de la información contenida en el formulario. Sin embargo, no es objetivo de este curso ahondar en el lenguaje *Javascript*, y sus diversos usos.

Ejemplo 3.4.1. A continuación se muestra un formulario formado por algunos de los tipos de campo descritos con anterioridad y se guarda en un archivo llamado "*encuesta.php*".

```
<html>
  <head>
    <title></title>
  </head>
  <body>
<form name="encuesta" method="get" action="verificar.php">
```

```
        NOMBRE <input type="text" name="nombre" size="43"><br>
        <br>
        NIVEL DE INTERNET<br>
        bajo <input type="radio" name="nivel" value="bajo" checked>
        medio <input type="radio" name="nivel" value="medio">
        alto <input type="radio" name="nivel" value="alto"><br>
        <br>
        EXPERIENCIA PREVIA EN PROGRAMACIÓN<br>
        Visual Basic <input type="checkbox" name="basic">
        C/C++ <input type="checkbox" name="c_cplus">
        Java <input type="checkbox" name="java"><br>
        <br>
        TU OPINIÓN SOBRE ESTE CURSO<br>
        <textarea name="opinion" cols="40" rows="5"></textarea><br>
        <br>
        <input type="submit" value="Enviar">
        <input type="reset" value="Borrar todo">
    </form>
   </body>
</html>
```

Deberíamos ver un formulario similar a este:



Recepción de datos de formularios.

Recibir los datos no puede ser más sencillo. El script de destino tendrá definida un variable por cada campo con el mismo nombre que se le haya dado en el formulario. Y esta variable contendrá el valor que se haya introducido en el campo. Esta variable ya está disponible desde el comienzo del script, no hay que hacer nada especial, sólo leer su contenido.

Ejemplo 3.4.2. Los parámetros que nos han llegado se encuentran almacenados en un array asociativo con nombre de variable $_REQUEST. Podemos ver su contenido usando la función *print_r()* que se vió en la unidad didáctica anterior. Aunque en algún caso se puede acceder a estos datos sin $_REQUEST, resulta totalmente desaconsejado ya que su efectividad dependerá del navegador[1].

```
<pre> <?php print_r($_REQUEST); ?> </pre>
```

Ejemplo 3.4.3. Esta página *PHP* recibe los datos de la encuesta y los muestra. Para que funcione hay que guardarla con el nombre "*verificar.php*", que es el que aparecía en el atributo *action* del formulario.

```
<?php
  echo "Comprueba si tus datos son correctos.<br>";
  echo "<br>";
  echo "{$_REQUEST["nombre"]}.<br>";
  echo "Nivel de internet: {$_REQUEST["nivel"]}.<br>";
  if (($_REQUEST["basic"] == "on")
       || ($_REQUEST["c_cplus"] == "on")
       || ($_REQUEST["java"] == "on")) {
    echo "Con experiencia en ";
    if ($_REQUEST["basic"] == "on") {
      echo "Visual Basic";
      if ($_REQUEST["c_cplus"] == "on") {
        echo ", C/C++";
      }
      if ($_REQUEST["java"] == "on") {
        echo ", Java";
      }
    } else if ($_REQUEST["c_cplus"] == "on") {
      echo "C/C++";
      if ($_REQUEST["java"] == "on") {
        echo ", Java";
      }
    } else {
      if ($_REQUEST["java"] == "on") {
        echo "Java";
      }
    }
    echo ".<br>";
  } else {
    echo "Sin experiencia previa en programación.<br>";
  }
  echo "<br>";
  echo "OPINIÓN SOBRE EL CURSO:<br>";
  echo nl2br($_REQUEST["opinion"]); //convierte saltos de línea en <br>
?>
```

Páginas que envían, reciben y procesan datos.

En este apartado se muestra una práctica común mediante la que la página web que contiene el formulario, y la página web que recibe y procesa la información se fusionan en un mismo fichero, haciendo uso por tanto de una única página web.

Para aplicar esta técnica debemos tener en cuenta los siguientes aspectos:

- Debemos comprobar si recibimos alguno de los datos de formulario:
- En caso de recibir alguno de los datos, los procesaremos.

- Si no recibiéramos los datos, mostraremos el formulario.

Debemos enviar el formulario, mediante su atributo *action*, a la propia pagina web que contiene el formulario. Para ello, podemos emplear la variable predefinida ***$_SERVER['PHP_SELF'];***, que contiene el nombre de archivo de la página web actual.

NOTA: El array superglobal $_SERVER es un array que contiene información, tales como cabeceras, rutas y ubicaciones de script. Las entradas de este array son creadas por el servidor web. No hay garantía que cada servidor web proporcione alguna de estas entradas. Para saber más
https://www.php.net/manual/es/reserved.variables.server.php

Sin embargo, esta práctica tan extendida tiene unas ventajas y unos inconvenientes que pueden no resultar siempre beneficiosos. Por un lado tenemos la ventaja de agrupar conceptualmente toda la acción en un único fichero, ya que con una única página web podremos contener todos los elementos necesarios para llevar a cabo una tarea. Sin embargo, este procedimiento dificulta considerablemente la separación de la programación y el diseño.

Por este motivo, resulta desaconsejable llevar a cabo esta práctica en aplicaciones que vayan a tener una envergadura considerable.

Ejemplo 3.4.4. Esta página web une el formulario de entrada de datos y la recepción de los mismos. Podemos guardar este archivo con el nombre "*juegoppt.php*".

```
<?php
$opciones = array("piedra", "papel", "tijeras");
if (isset($_REQUEST["jugada"] )) {
    $mijugada = $opciones[rand(0, 2)];
    if ($_REQUEST["jugada"] == $mijugada) {
        $resultado = "Empates.";
   } else if (($_REQUEST["jugada"]=="piedra" && $mijugada == "tijeras")
||
      ($_REQUEST["jugada"] == "tijeras" && $mijugada == "papel")||
      ($_REQUEST["jugada"] == "papel" && $mijugada == "piedra")) {
            $resultado = "Tu ganas.";
    } else {
            $resultado = "Gano yo.";
    }
}
?>

<html>
<head>
<title>Piedra, papel o tijera</title>
</head>
<body>
<?php
echo isset($_REQUEST["jugada"])?"Has elegido {$_REQUEST['jugada']}, yo
he elegido $mijugada $resultado <br>¿Quieres jugar otra vez?<br>":"";
?>
<form "juego" method="post" action="<?= $_SERVER['PHP_SELF'] ?>">
Piedra<input type="radio" name="jugada" value="piedra">
Papel<input type="radio" name="jugada" value="papel">
Tijera<input type="radio" name="jugada" value="tijeras">
<input type="submit" value="Jugar">
</form>
</body>
</head>
```

El código que se incluye al principio sólo se ejecuta si el dato *$jugada*

no llega vacío. Es decir, cuando se ha pulsado el botón de enviar.

Redirección de páginas web.

En ocasiones se necesita que un programa *PHP* reciba datos y haga una operación "silenciosa" con los mismos. Es decir, que no sea necesario que se muestre una página al navegador sino que se vuelva a la inicial, como un formulario que envía, recibe y procesa datos.

Una alternativa es hacer una página que procese los datos y que al terminar se redirija automáticamente a otra. Para poder hacerlo *PHP* nos ofrece la función *header()*, que sirve para enviar cabeceras *HTML*, y que en el caso concreto que nos ocupa, se usa de la siguiente forma:

```
header("location:url")
```

La dirección de destino *url* puede ser una ruta absoluta o relativa.

Es imprescindible que la página no contenga al principio código *HTML*, ya que las cabeceras deben enviarse siempre al principio del documento. De hecho, aquí nos encontramos un error muy habitual, ya que tan sólo con que se nos haya descuidado un espacio o salto de linea a enviar como *HTML*, ocasionará un error.

También hay que tener en cuenta que una vez que se llega a la función *header* no se sigue ejecutando el resto de la página, por lo que las operaciones que queramos hacer las deberemos ejecutar antes.

Un uso muy habitual del *header location* es realizar *get* implícitos de manera automática.
Por ejemplo:

```
$usuario='Juan';
$password='1234';
$usu=$_GET['usu'];
$pass=$_GET['pass'];

if($usuario==$usu && $password==$pass)
   header ('location:inicio.php?usuario='.$usuario);
else
   header ('location:login.php?error=Usuario o contraseña incorrectos');
```

Aquí iremos o a la página de inicio de la aplicación enviando el usuario o a la página de login donde mostramos un error.

Ejemplo 3.4.5. Este es un ejemplo típico de recepción de datos. En primer lugar, se incluye una sencilla página web con un formulario que pide el nombre y un comentario para almacenarlo en un registro de visitas. En el script "*insertar.php*" se almacena en la base de datos y se redirecciona a una página para dar las gracias.

visita.html

```
<html>
  <head>
    <title>Libro de visitas</title>
  </head>
  <body>
```

```
    <form          name="visita"          method="post"
      action="insertar.php">       Nombre:       <input
      type="text"     name="nombre"     size="44"><br>
      Comentario:<br>
      <textarea name="comentario" cols="40" rows="5"></textarea><br>
      <input type="submit" value="Enviar">
    </form>
  </body>
</html>
```

insertar.php

```
<?php
  // Aquí vendría el código para guardar en la base de datos
  header("location:gracias.html"); //Redirección
?>
```

gracias.html

```
<html>
  <head>
    <title>Gracias</title>
  </head>
  <body>
    Gracias por su participación.
  </body>
</html>
```

Ejemplo 3.4.6. Uno de sus usos habituales es redirigir hacia una página de error, en caso de que haya fallos en los datos. Aquí se valida una dirección de email, y si es incorrecta se reenvía a una página de error.

suscripcion.html

```
<html>
<head>
  <title>Suscripción al boletín electrónico</title>
</head>
<body>
  Bienvenido al servicio de suscripción.<br>
  Por favor, introduzca una dirección de email válida.<br>
  <form name="suscripcion" method="post" action="guardar.php">
    Email: <input type="text" name="email" size="40"><br>
    <input type="submit" value="Enviar">
  </form>
</body>
</html>
```

guardar.php

```
<?php
  // Indica la posición del caracter "@" o FALSE si no está
  $posicion_arroba = strpos($_REQUEST["email"], "@");
  // Busca la aparición de un punto (.) a partir de la arroba
  $posicion_punto = strpos($_REQUEST["email"], ".", $posicion_arroba);
  if ($posicion_arroba && $posicion_punto) {
    // Aquí vendría el código para guardar en la base de datos
    header("location:confirmacion.html"); // Redirección
  } else {
    // Aquí vendría el código para guardar en la base de datos
```

```
        header("location:error.html");    // Redirección
      }
    ?>
```

confirmacion.html

```
    <html>
      <head>
        <title>Confirmacion</title>
      </head>
      <body>
        Su     suscripción     ha     sido
        registrada.</br>  Gracias  por  su
        interés.
      </body>
    </html>
```

error.html

```
    <html>
      <head>
        <title>Error</title>
      </head>
      <body>
        Introdujo  una  dirección  de  correo  no
        válida.<br> Por favor vuelva a solicitar la
        suscripción.
        <a href="suscripcion.html"> Volver </a>
      </body>
    </html>
```

Nuevos Operadores Ternarios

PHP ha introducido 3 operadores Ternarios muy potentes. El uso de estos operadores aumenta la productividad, menos líneas y más limpias.

NOTA: php 8.5 ha introducido el Operador Pipe (**|>**) , pero por reciente, no lo explico en esto apuntes y lo dejo como opcional de investigación.

1. El Operador Ternario Tradicional (? :)

Este ya lo conocemos. Es una forma corta de escribir un if/else. Evalúa una condición y devuelve un valor si es verdadera, y otro si es falsa. Esta parte es un repaso

* **Sintaxis:** condición ? valor_si_verdadero : valor_si_falso;

```
$edad = 20;
// Con if/else tradicional:
if ($edad >= 18) {
    $resultado = "Mayor de edad";
} else {
    $resultado = "Menor de edad";
}

// Con Operador Ternario:
$resultado = ($edad >= 18) ? "Mayor de edad" : "Menor de edad";

echo $resultado; // Imprime: Mayor de edad
```

Más ejemplos:

```
if(isset($_GET["limite"])){
  $limite = $_GET["limite"];
}else{
  $limite = 10;
}

hacerConsultaDeUsuariosConLimite($limite);
```

Nueva Versión

```
$limite = isset($_GET["limite"]) ? $_GET["limite"] : 10;
hacerConsultaDeUsuariosConLimite($limite);
```

2. Operador de Fusión de Null (??)

Los operadores ternarios, de fusión de null también denominado seguros contra nulos son herramientas clave en PHP para escribir código más limpio, corto y fácil de leer. Nos permiten evitar estructuras largas como los bloques if/else.

Este operador se diseñó específicamente para comprobar si una variable está definida y no es null. Es ideal para recibir datos de formularios o configuraciones sin que PHP lance un aviso (Warning) si la variable no existe.

* **Sintaxis:** variable ?? valor_por_defecto;

```
// Supongamos que $_GET['color'] no se ha enviado en la URL
$color_elegido = $_GET['color'] ?? 'azul';

echo $color_elegido; // Imprime: azul (porque $_GET['color'] no existía)
```

Este operador viene a remplazar los if, isset y tal vez los operadores ternarios. Lo que hace este operador es devolver el primer valor que no sea nulo.

Pongamos el ejemplo básico en el que nos mandan por medio de GET el límite de usuarios que queremos ver.

Si no mandan el límite, entonces asumimos un límite por defecto, que sería 10 (en este caso). Entonces lo haríamos así:

```
if(isset($_GET["limite"])){
  $limite = $_GET["limite"];
}else{
  $limite = 10;
}

hacerConsultaDeUsuariosConLimite($limite);
```

Si queremos ser un poco más elegantes, usamos el operador ternario y quedaría rescrito así:

```
$limite = isset($_GET["limite"]) ? $_GET["limite"] : 10;
hacerConsultaDeUsuariosConLimite($limite);
```

Pero igual no me gusta, mejor lo hacemos así:

```
$limite = $_GET["limite"] ?? 10;
hacerConsultaDeUsuariosConLimite($limite);
```

Así se ve algo mejor. En este momento no se nota mucho la diferencia, pero sigue leyendo para que veas cuán útil es.

Fusion null. Más de dos operandos

Siguiendo el ejemplo de arriba, ahora supongamos que modificamos nuestra API y aceptamos el límite tanto por POST así como por GET.

Normalmente quedaría así:

```
if(isset($_GET["limite"])){
  $limite = $_GET["limite"];
}else if(isset($_POST["limite"])){
  $limite = $_POST["limite"];
}else{
  $limite = 10;
}
hacerConsultaDeUsuariosConLimite($limite);
```

Con el operador ternario, quedaría así:

```
$limite = isset($_GET["limite"]) ? $_GET["limite"] :
isset($_POST["limite"]) ? $_POST["limite"] : 10;
hacerConsultaDeUsuariosConLimite($limite);
```

Finalmente, con este nuevo operador queda de la siguiente manera:

```
$limite = $_GET["limite"] ?? $_POST["limite"] ?? 10;
hacerConsultaDeUsuariosConLimite($limite);
```

Primero evalúa si hay algo en GET, luego en POST y finalmente, si ambos valores son nulos, se obtiene 10.

NOTA: Cierto es que se podría usar el `$_REQUEST` y en vez del código anterior podríamos hacer lo siguiente

```
$limite = isset($_REQUEST["limite"]) ? $_REQUEST["limite"] : 10;
hacerConsultaDeUsuariosConLimite($limite);
```

Pero el ejemplo de GET y POST es para ilustrar el funcionamiento del Operador coalescing null.

3, 4, 5 o más operandos

Nos venimos arriba puesto que se puede evaluar una lista infinita de operandos. Aquí un ejemplo:

```
$limite = null ?? null ?? null ?? 10 ?? 20 ?? 30;
```

En este caso, límite es 10, ya que es **el primer valor que no es nulo** y ahí deja de evaluar:

También trabaja bien con funciones:

```
function hola(){
  return "Hola mundo";
}

function adios(){
  return null;
```

```
}

$saludo = adios() ?? hola();
En este caso, saludo es “Hola mundo”:
```

Así como **podemos llamar a funciones** creadas por nosotros, podemos llamar a funciones nativas de PHP y claramente pasar argumentos.

Ejemplo final

Siguiendo nuestra ejemplos de usuarios en donde pueden mandar el límite por GET o POST.

Tendremos una función que devolverá los usuarios, pero que también puede devolver null. Y si devuelve null, devolvemos un array vacío. Juntando todo…

```
hacerConsultaDeUsuariosConLimite($_GET["limite"] ??
                  $_POST["limite"] ?? 10) ?? [];
```

Llamamos a la función ya sea con lo que haya en GET, POST o con el número 10. Y si nos devuelve null, entonces devolvemos []; es decir, un array vacío.

Básicamente este operador simplifica el uso de isset y el encadenamiento largo con el operador ternario. Devuelve el primer argumento que no sea nulo, y podemos evaluar una lista infinita de operandos.

Fusión de Null Compuesta (??=)

Es una ampliación del fusion Null y permite asignar el valor por defecto directamente a la variable original si esta es nula.

```
$configuracion['timeout'] = null;

// Si es null, le asigna 30 automáticamente
$configuracion['timeout'] ??= 30;

echo $configuracion['timeout']; // Imprime: 30
```

3. Operador Seguro contra Nulos (?->)

Disponible desde PHP 8.0, este operador sirve para llamar a métodos o propiedades de un objeto sin arriesgarte a que el código falle si el objeto resulta ser null. Si el objeto es nulo, la cadena se detiene y devuelve null en lugar de romper la aplicación.

```
// Supon función que busca usuario,puede no encontrarlo y devolver null
$usuario = buscarUsuario(15);

// Sin ?-> (Peligroso si $usuario es null, daría error fatal)
// Sin devuelve null el siguiente código falla
// $ciudad = $usuario->getPerfil()->getCiudad();
```

Podríamos solucionarlos con if:

```
$ciudad = null;

// Comprobamos paso a paso de forma segura
if ($usuario !== null && is_object($usuario)) {
```

```
    $perfil = $usuario->getPerfil();

    if ($perfil !== null && is_object($perfil)) {
        $ciudad = $perfil->getCiudad();
    }
}
```

Es correcto pero, mira que fácil y elegante con $->

```
// Con ?-> (Seguro)
$ciudad = $usuario?->getPerfil()?->getCiudad();
// Si el usuario no existe, $ciudad valdrá null de forma segura.
// Vamos que si vale null no llama a getCiudad y no falla
```

Un poco de Seguridad en el Envío de Datos. Sanitización de datos.Limpiar y validar variables GET y POST en PHP

Limpiar y validar variables GET. Si estás trabajando en un sistema seguramente se han encontrados con formularios que trabajan con métodos GET y POST para manejar la información del sistema o quizá al momento de editar in registro usan el identificador del registro para pasarlo a la URL y mostrar tal registro.

En estos casos es de vital importancia limpiar las variables de posibles inyecciones SQL que pudiéramos ser víctimas y posterior evitarnos dolores de cabeza ante accesos no autorizados en nuestra plataforma.

¿Cuál es la solución para estos problemas?

El método es muy sencillo de aplicar y solo se debe agregar al inicio de nuestros ficheros ".php" que se van a usar en el proceso. Sin embargo, el trabajo conciso de este método es recorrer todos los valores recibidos por GET y POST y limpiarlos de cadenas peligrosas o validar que los datos sean correctos.

La función **filter_var s**e puede utilizar para realizar dos acciones distintas:

- Validar Datos
- Sanitizar Datos

Validación y desinfección de datos con filtros

Limpiar y validar la entrada del usuario es una de las tareas más comunes en una aplicación web. Para facilitar esta tarea, PHP proporciona una extensión de filtro nativa que puede utilizar para desinfectar o validar datos como direcciones de correo electrónico, direcciones URL, direcciones IP, números, cadenas de texto, etc.

Para validar los datos utilizando la extensión de filtro es necesario utilizar la función **filter_var()** de PHP. La sintaxis básica de esta función se puede dar con:

Sintaxis

filter_var(*variable*, *filter*, *options*)

Esta función toma tres parámetros de los cuales los dos últimos son opcionales. El primer parámetro es el valor que se va a filtrar, el segundo parámetro es el identificador del filtro que se va a aplicar y el tercer parámetro es la matriz de opciones relacionadas con el filtro. Veamos cómo funciona.

La diferencia entre validar o desinfectar/sanitizar, depende simplemente del parámetro **OPTIONS.**

Por cierto, validar todo sabemos que significa. En este contexto si validamos devolverá false si no cumple la condición o el contenido de la variable si sí la cumple

Pero, ¿qué significa sanitizar? La sanitización sirve para limpiar de caracteres peligrosos la variable. Cuando decimos peligros son aquellos que puedan ser utilizados para "atacar" al servidor.

Por ejemplo podríamos rellenar en un campo text con el siguiente contenido:

<script> Codigo malicioso</script>

Sanitizar sirve para "limpiar", "evitar" este tipo de ataques.

Ejemplo de "desinfección": El ejemplo siguiente se desinfectará una cadena quitando todas las etiquetas HTML de ella:

a) Ejemplo de limpiar/sanitizar texto

```
<?php
$comentario = "<h1>Hola! Buenos dias con todos</h1>";

// Desinfectar e imprimir cadena de comentarios
$limpiacomentario= filter_var($comentario , FILTER_SANITIZE_STRING);
echo $limpiacomentario;
?>
```

La salida del ejemplo anterior se verá algo como esto:

```
Hola! Buenos dias con todos
```

Simplemente con este ejemplo, ya nadie podría hacer el ataque anteriormente visto.

b) Sanitizar y Validar valores enteros

¿Como sabemos que "limpia"/"sanitiza" datos? Sencillo, fijate en el parámetro **FILTER_SANITIZE_STRING** . Dicho parámetro indica que el texto pasado va a ser limpiado de caracteres que puedan ser usados para realizar algún tipo de ataque al servidor.

b) Sanitizar y Validar valores enteros

En el ejemplo siguiente se valida si el valor es un entero válido o no.

```
<?php
// Ejemplo de variable numerica no valida
$novalido = "//25[]";

// Quitamos los caracteres ilegales de la variable
$numero = filter_var($novalido, FILTER_SANITIZE_NUMBER_INT);

// Validamos la variable filtrada
if(filter_var($numero, FILTER_VALIDATE_INT)){
echo "EL $numero ES NUMERO VALIDO";
} else{
echo "EL $numero NO ES NUMERO VALIDO";
}
?>
```

La salida en el navegador:

```
EL 25 ES NUMERO VALIDO
```

En este caso realizamos las dos acciones, primero limpiamos y una vez limpio el número, comprobamos que es un entero valido.

Distinguir cada acción es sencillo, fijate en los parametros:

- **FILTER_SANITIZE_NUMBER_INT**
- **FILTER_VALIDATE_INT**

Validar también devuelve la variable si es un entero en este caso o false, si no es entero

NOTA: Recuerda que en php todo lo que no sea false, es verdadero, por tanto aunque `filter_var($numero, FILTER_VALIDATE_INT)` devuelve 25, se considera como NO FALSE y por tanto se cumple la condición del if.

Lista de filtros de sanitización

La lista completa la podemos obtener de la siguiente página
https://www.php.net/manual/es/filter.filters.sanitize.php

Alguno ejemplos que podríamos usar son:
FILTER_SANITIZE_EMAIL.– Elimina todos los caracteres menos letras, dígitos y *!#$%&'*+-=?^_`{|}~@.[]*.
FILTER_SANITIZE_ENCODED.– String URL-encode, opcionalmente elimina o codifica caracteres especiales.
FILTER_SANITIZE_NUMBER_INT.– Elimina todos los caracteres excepto dígitos y los signos de suma y resta.
FILTER_SANITIZE_STRING.- Elimina etiquetas, opcionalmente elimina o codifica caracteres especiales.

Lista de filtros de Validación

La lista completa la podemos obtener de la siguiente página
https://www.php.net/manual/en/filter.filters.validate.php

Por último, OLVIDATE de is_number y esas funciones, USA filter_var.

Aquí tienes un ejemplo completo

```
<?php
declare(strict_types=1);
// 1. Definimos las funciones de limpieza individuales
function limpiar_texto(string $texto): string
{
    return strip_tags(trim($texto));
}

function convertir_minusculas(string $texto): string
{
    return strtolower($texto);
}

// 2. Función principal que procesa la superglobal $_REQUEST
function procesar_solicitud(array $request): ?array
{
    // Verificamos campos obligatorios usando el operador de fusión de
PHP (??)
    $rawUsername = $request['username'] ?? null;
    $rawEmail    = $request['email'] ?? null;
    $rawEdad     = $request['edad'] ?? null;
    $rawPassword = $request['password'] ?? null;

    if (!$rawUsername || !$rawEmail || !$rawPassword) {
        return null; // Faltan campos requeridos
    }

    // Validamos el formato del email antes de limpiarlo
    if (!filter_var($rawEmail, FILTER_VALIDATE_EMAIL)) {
        return null; // Correo electrónico con formato totalmente
inválido
    }

    // --- PROCESO DE SANITIZACIÓN TRADICIONAL EN PHP 8 ---
```

```
// Paso a paso para el Username (Evita anidar funciones como:
convertir_minusculas(limpiar_texto($rawUsername)))
    $usernameLimpio = limpiar_texto($rawUsername);
    $usernameLimpio = convertir_minusculas($usernameLimpio);

    // Paso a paso para el Email usando los filtros nativos de PHP
    $emailLimpio = trim($rawEmail);
    $emailLimpio = (string) filter_var($emailLimpio,
FILTER_SANITIZE_EMAIL);
    $emailLimpio = convertir_minusculas($emailLimpio);

    // Filtrado de edad para asegurar que sea un número entero
    $edadFiltrada = filter_var($rawEdad, FILTER_VALIDATE_INT);
    $edadFinal = ($edadFiltrada !== false) ? $edadFiltrada : 0;

    // Retornamos un array limpio listo para ser usado en el sistema
    return [
        'username' => $usernameLimpio,
        'email'    => $emailLimpio,
        'edad'     => $edadFinal,
        'password' => password_hash($rawPassword, PASSWORD_BCRYPT) //
Hash seguro de la clave
    ];
}
// --- SIMULACIÓN DE DATOS DE ENTRADA EN $_REQUEST ---
$_REQUEST['username'] = "  <script>alert('ataque')</script> AnaGomez77
";
$_REQUEST['email']    = "   ANA.GOMEZ@eMaIL.coM   ";
$_REQUEST['edad']     = "34";
$_REQUEST['password'] = "MiClaveSuperSegura456";

// Ejecutamos el script pasando la superglobal
$datosLimpios = procesar_solicitud($_REQUEST);

if ($datosLimpios !== null) {
    echo "<h3>Datos de \$_REQUEST procesados con éxito (Sin objetos ni
Pipe):</h3>";
    echo "<b>Username final:</b> " . $datosLimpios['username'] . "<br>";
// anagomez77
    echo "<b>Email final:</b> " . $datosLimpios['email'] . "<br>";
// ana.gomez@email.com
    echo "<b>Edad final:</b> " . $datosLimpios['edad'] . "<br>";
// 34
} else {
    echo "Error: Los datos enviados no son válidos o están
incompletos.";
}
```

Vamos a lo práctico: filter_var_array

El **filter_var_array** me permite de una manera más cómoda asignar los distintos filtros a los datos que obtengo después del envío de los mismos:

Mismo ejemplo anterior con filver_var_array

```
<?php

declare(strict_types=1);

/**
 * Función tradicional para sanitizar el nombre de usuario.
 * Quita etiquetas HTML, limpia espacios y pasa todo a minúsculas.
 */

function sanitizar_nombre_usuario(mixed $valor): string
{
    $cadena = (string) $valor;
    $limpio = strip_tags(trim($cadena));
    return strtolower($limpio);
}

/**
 * Procesa y limpia la superglobal $_REQUEST utilizando un mapa de
filtros.
 */
function procesar_solicitud_con_array(array $request): ?array
{
    // 1. Definimos la configuración y las reglas para cada campo de
$_REQUEST
    $reglasFiltros = [
        'username' => [
            'filter' => FILTER_CALLBACK,
            // Pasamos el nombre de la función tradicional como un
string/callable
            'options' => 'sanitizar_nombre_usuario'
        ],
        'email' => [
            // Valida que el formato del correo sea correcto
            'filter' => FILTER_VALIDATE_EMAIL,
            // Si el correo es inválido, devuelve NULL en lugar de FALSE
            'flags'  => FILTER_NULL_ON_FAILURE
        ],
        'edad' => [
            'filter' => FILTER_VALIDATE_INT,
            'options' => [
                // Rango de edad permitido
                'min_range' => 18,
                'max_range' => 120
            ],
            // Si no cumple el rango numérico, devuelve NULL
            'flags' => FILTER_NULL_ON_FAILURE
        ],
        'password' => [
            // Mantiene el valor original sin alterarlo para no dañar
caracteres especiales
            'filter' => FILTER_DEFAULT
        ]
    ];

    // 2. Ejecutamos el filtro masivo sobre la superglobal $_REQUEST
```

```
    $datosFiltrados = filter_var_array($request, $reglasFiltros, false);

    if (!$datosFiltrados) {
        return null;
    }

    // 3. Validaciones obligatorias de negocio post-filtrado
    if (empty($datosFiltrados['username']) || $datosFiltrados['email']
=== null || empty($datosFiltrados['password'])) {
        return null;
    }

    // Si la edad no cumplió el rango (quedó en NULL), le asignamos 0
por defecto
    if ($datosFiltrados['edad'] === null) {
        $datosFiltrados['edad'] = 0;
    }

    // 4. Formateo y transformaciones de seguridad finales
    $emailLimpio = (string) filter_var($datosFiltrados['email'],
FILTER_SANITIZE_EMAIL);
    $datosFiltrados['email'] = strtolower(trim($emailLimpio));

    // Encriptamos la contraseña de manera segura
    $datosFiltrados['password'] = password_hash((string)
$datosFiltrados['password'], PASSWORD_BCRYPT);

    return $datosFiltrados;
}

// --- SIMULACIÓN DE DATOS DE ENTRADA EN $_REQUEST ---

$_REQUEST['username'] = "  <script>alert('xss')</script> PedroPicapiedra
";
$_REQUEST['email']    = "   PEDRO.P@eMaIL.coM   ";
$_REQUEST['edad']     = "35";
$_REQUEST['password'] = "ClaveValida987!";

// Ejecutamos el script pasando la superglobal
$resultadoFinal = procesar_solicitud_con_array($_REQUEST);

if ($resultadoFinal !== null) {
     echo "<h3>Datos de \$_REQUEST procesados:</h3>";
   echo "<b>Username final:</b> " . $resultadoFinal['username'] .
"<br>"; // pedropicapiedra
    echo "<b>Email final:</b> " . $resultadoFinal['email'] . "<br>";
// pedro.p@email.com
    echo "<b>Edad final:</b> " . $resultadoFinal['edad'] . "<br>";
// 35
} else {
    echo "Error: Los datos enviados no superaron las reglas de
validación.";
}
```

Ventajas clave de filter_var_array:

- **Declarativo y centralizado:** Toda la lógica de lo que esperas recibir se lee en un solo array asociativo de configuración ($reglasFiltros).
- **Ahorro de validaciones isset() o ??:** Si un parámetro no viene en $_REQUEST, filter_var_array() lo gestiona internamente basándose en la configuración del tercer parámetro.
- **Control de errores elegante con Flags:** El uso de FILTER_NULL_ON_FAILURE permite normalizar las respuestas fallidas de los filtros, haciendo que sea muy sencillo validar después si el dato es útil o no con un simple chequeo de === null

Resumiendo, con **filter_var o filter_var_array** conseguimos dos objetivos:

1. Validar datos. Olvidate de is_number y esas funciones.
2. Aumentamos la seguridad limitando los ataques de Cross Site Scription, file upload, file inclusión, CSRF… Esto es debido a que "**negamos**" los caracteres que permiten la ejecución de los comandos que se necesitan para dichos ataques. ¿Cómo de seguro es esto? Si bien la seguridad absoluta no existe, la sanitización de datos es una herramienta fundamental y clave para mejorar el nivel de seguridad de nuestras aplicaciones.

Subida de ficheros al Servidor.

El envío de archivos desde el cliente es una de las posibilidades más atractivas para una aplicación web. Se pueden subir de esta forma fotos, un curriculum, un documento PDF, etc.

Para poder enviar documentos en un formulario necesitamos dos elementos:

- Incluir el atributo ***enctype*** con el valor ***multipart/form-data*** en el formulario de envío.
- Incluir un campo input del tipo file al que el cliente asociará el fichero que enviaremos en el formulario.

Ejemplo 3.5.9. Se va a explicar como subir un archivo a través de un ejemplo. En primer lugar necesitamos un formulario especial, que llamaremos "*subir.html*".

subir.html

```
<html>
  <head>
    <title>Elija el archivo a subir</title>
  </head>
  <body>
    <form name="subir" method="post" action="guardar_fichero.php"
                                enctype="multipart/form-data">
      Archivo a subir <input type="file" name="miarchivo"><br>
      <input type="submit" value="Subir el archivo">
    </form>
  </body>
</html>
```

Además de este formulario necesitaremos una página dinámica en *PHP* que reciba el archivo y lo guarde en el disco duro. Esto se hace en 3 pasos:

1. El servidor guarda el archivo en un fichero temporal.
2. Movemos el archivo en la posición definitiva usando la función *move_uploaded_file()*. Esto es imprescindible ya que pasado cierto tiempo el servidor web eliminará automáticamente el fichero temporal.
3. Si la operación de mover el archivo falla mostramos un mensaje de error.

El código que se ha escrito en este caso, en el script ***guardar_fichero.php*** es:

guardar_fichero.php

```
<?php
  $temporal = $_FILES["miarchivo"]["tmp_name"];
  $destino = "uploads/" . $_FILES["miarchivo"]["name"];
  if (move_uploaded_file($temporal, $destino)) {
    echo "Archivo subido con éxito";
  } else {
    echo "Ocurrió un error, no se ha podido subir el archivo";
  }
?>
```

Hay que tener cuidado, ya que en este script se asume que el directorio *uploads* ya existe. Si no es así, la operación fallaría.

Fíjese que para acceder al fichero, *PHP* nos crea un array superglobal denominado ***$_FILES***, que contendrá todos los ficheros que ha enviado el usuario. De este modo, mediante el nombre del

campo del fichero, que puede consultarse en el formulario correspondiente, podría acceder a cada uno de los ficheros enviados para realizar las acciones oportunas.

Array Superglobal $_FILES

El array superglobal $_FILES se crea en el servidor al subir uno o varios archivos. En él, disponemos información del fichero o ficheros subidos. El contenido de los atributos de cada archivo es el siguiente:

- **name**: Nombre original del fichero.
- **type**:tipo MIME del archivo subido. Para ver todos los MIME disponibles, pincha aquí: https://www.htmlquick.com/es/reference/mime-types.html
- **tmp:** El servidor le da un nombre temporal al fichero subido. Este es su nombre.
- **error:** Si se ha subido correctamente o no
- **size:** Tamaño del fichero subido

Por supuesto si se suben varios ficheros cada elemento genera una posición en array $_FILES para cada fichero cada uno con estos campos.

NOTA: El manejo del envío de ficheros se puede configurar en el fichero php.ini con las directivas file_uploads, upload_max_filesize, upload_tmp_dir, post_max_size y max_input_time.

Subida de múltiples ficheros

Para subir múltiples ficheros tenemos que modificar un formulario para que queda de la siguiente manera:

```
<form action="index.php" method="post" enctype="multipart/form-data">
  <input name="upload[]" type="file" multiple="multiple" />
</form>
```

Puntos importantes: el ***enctype="multipart/form-data"*** en la etiqueta *form* para que este envíe correctamente los archivos que subimos, el definir el nombre del input como un array poniendo los **corchetes *[]*** y la etiqueta ***multiple='multiple'*** que nos permitirá seleccionar varios ítems a las vez.

Vamos ahora con el **PHP** con explicación línea a línea:

```
// RECORREMOS LOS FICHEROS
if(isset($_FILES)){
 for($i=0; $i<count($_FILES['upload']['name']); $i++) {
  //Obtenemos la ruta temporal del fichero
    $fichTemporal = $_FILES['upload']['tmp_name'][$i];

   //Definimos una ruta definitiva para guardarlo
    $destino = "./nuestraCarpeta/" . $_FILES['upload']['name'][$i];

    //Movemos a la ruta final
    if(move_uploaded_file($fichTemporal, $destino)) {
       //imprimimos el nombre del archivo subido
      echo"Se ha subido el fichero ".$_FILES['upload']['name'][$i];
     }
   }
 }
}
```

Más ejemplos

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de subir fichero</title>
</head>
<body>
<form  action="subirfichero.php"  method="post"  enctype="multipart/form-
data">
Fichero: <input type="file" name="fichero" />
<br />
<input type="submit" value="Enviar" />
</form>
</body>
</html>
```

El segundo fragmento de código es la página PHP que recibe el fichero enviado en el servidor. En este código se emplea la función file_exists(nombre) para comprobar si ya existe el fichero que se intenta subir y la función move_uploaded_file(origen, destino) para mover el fichero temporal de la subida a su destino definitivo.

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de subir fichero</title>
</head>
<body>
<p>
<?php
 $msgError = array(0 => "No hay error, el fichero se subió con éxito",
               1 => "El tamaño del fichero supera la directiva
                   upload_max_filesize el php.ini",
               2 => "El tamaño del fichero supera la directiva
                   MAX_FILE_SIZE especificada en el formulario HTML",
               3 => "El fichero fue parcialmente subido",
               4 => "No se ha subido un fichero",
               6 => "No existe un directorio temporal",
               7 => "Fallo al escribir el fichero al disco",
                     8 => "La subida del fichero fue detenida por la
extensión");

 if($_FILES["fichero"]["error"] > 0)
 {
   echo "Error: " . $msgError[$_FILES["fichero"]["error"]] . "<br />";
 }
 else
 {
   echo "Nombre original: " . $_FILES["fichero"]["name"] . "<br />";
   echo "Tipo: " . $_FILES["fichero"]["type"] . "<br />";
     echo  "Tamaño:  "  .  ceil($_FILES["fichero"]["size"]  /  1024)  .  "
Kb<br />";
   echo "Nombre temporal: " . $_FILES["fichero"]["tmp_name"] . "<br />";

   if(file_exists("upload/" . $_FILES["fichero"]["name"]))
   {
    echo $_FILES["fichero"]["name"] . " ya existe";
   }
   else
```

```
   {
    move_uploaded_file($_FILES["fichero"]["tmp_name"],
                  "upload/" . $_FILES["fichero"]["name"]);
    echo "Almacenado en: " . "upload/" . $_FILES["fichero"]["name"];
   }
 }
?>
</p>
</body>
</html>
```